

IMPROVED EXPONENTS FOR 3SUM AND APSP FROM TRIANGLES IN SPARSE LOPSIDED GRAPHS

JIHOON HYUN

ABSTRACT. Alman and Vassilevska Williams gave deterministic algorithms for 3SUM in time $O(n^{1.9992})$ and for the $(\min, +)$ -product and APSP in time $O(n^{2.99942})$, through an algorithm for Exact Triangle in time $n^{3-0.00175}$. We analyze their algorithm again and change it in a few places. An exact count of the leaves of the recursion, a host that charges the solver at the true size of the middle part, and encodings restricted to the leaves that are read raise the saving for Exact Triangle from 0.00175 to every number below 0.002095, so that 3SUM is solved in time $O(n^{1.99896})$; we show that the supremum of the savings of the method is at most 0.002096. Following a footnote of the paper, we also solve the all-edges version of Exact Triangle within the same bound and reduce the $(\min, +)$ -product to it on the same number of vertices, so that APSP keeps the whole saving: the $(\min, +)$ -product and APSP are solved in time $O(n^{2.99791})$, against $O(n^{2.99931})$ through the standard reductions. All statements are about programs of a word RAM, hold without any hypothesis, and are machine-checked in Lean 4 on top of the formalization of the original paper; the pruned encoder is written as a program and its running time is proved on the machine.

CONTENTS

1. Introduction	1
2. The framework of Alman and Vassilevska Williams	3
3. The exact count of the leaves	5
4. Pruned encodings	8
5. Charging the solver at the true size of the middle part	11
6. The optimal parameters	13
7. Consequences for 3SUM and APSP	16
8. The formalization	20
9. Limits of the method	23
References	25

1. INTRODUCTION

Alman and Vassilevska Williams [1] gave the first truly subquadratic algorithm for 3SUM and the first truly subcubic algorithm for APSP. Both follow from a deterministic algorithm for Exact Triangle in time $n^{3-0.00175}$ up to a logarithmic factor, which rests on a fast algorithm for *thin* matrix products: a recursion with Schönhage’s identity for $\langle 3, 1, 3 \rangle \oplus \langle 1, 4, 1 \rangle$, pruned to the entries that are wanted. The savings are small, and the paper does not optimize them.

Date: October 7, 2026.

2020 Mathematics Subject Classification. Primary 68Q25; Secondary 68W40, 68V20.

Key words and phrases. 3SUM, all-pairs shortest paths, Exact Triangle, thin matrix products, Schönhage’s identity, formalized mathematics, Lean.

The result has since been formalized in Lean 4 as a set of statements about programs of a word RAM [2].

This paper asks how far the same method goes with the same base identity. It has three ingredients, a determination of the optimal parameters, and the all-edges route to APSP that footnote 10 of the paper mentions.

The ingredients. *An exact count of the leaves* (Section 3). The cost of the recursion is governed by the numbers β_d of leaves of each order in a tile. The paper bounds their tail by a geometric series. We bound it by its exponential rate, a function g_2 of the ratio $c = L/m$ of the levels and of the relative order (Theorem 3.3); the proof is a comparison of integers. The paper’s exponent is the tangent of the exact one at zero. The algorithm does not change: the same program gets a better bound on its running time (Theorem 3.5).

The true size of the middle part (Section 5). The reduction from Exact Triangle (Theorem 17 of the paper) produces instances whose middle part has about D'/g' vertices, where D' is the dimension parameter and g' the number of pieces per hash class, and calls the solver as if it had D' . Passing the true size changes one assignment of the program (Theorem 5.1). The solver then runs at a smaller inner dimension D and, relative to it, with fewer wanted entries: n^2/D^σ with $\sigma > 1/2$ instead of $n^2/\sqrt{D}$. Equivalently, the weights are hashed modulo a larger prime (Remark 5.4).

Pruned encodings (Section 4). The recursion reads the encodings of the input only at the leaves with at most m symbols P_0 . They can be computed by a pruned form of Yates’ algorithm in $O(LM)$ operations per band instead of $O(10^L)$ (Theorem 4.2), which relaxes the condition under which a product counts as thin. We write the pruned encoder as a program and prove its running time on the machine (Theorem 4.6); the rest of the solver is upstream’s, re-verified under the weaker contract that the encodings are right only at the leaves that are read.

APSP through all-edges Exact Triangle (Section 7.4). The known reductions give the $(\min, +)$ -product and APSP a third of the saving of Exact Triangle. Footnote 10 of the paper observes that the host of its reduction also solves the all-edges version of Exact Triangle, to which the $(\min, +)$ -product reduces on the same number of vertices. We write the two programs (the all-edges host and the reduction, Theorems 7.9 and 7.8), and APSP keeps the whole saving.

The optimum. With these ingredients the saving for Exact Triangle depends on four parameters. For a fixed ratio c its supremum has a closed form: it is $R(\Gamma)\Gamma/2$, where $\Gamma = \Gamma(c)$ is the value at which two explicit curves cross and R is the bound of the thinness condition (Theorem 6.2). With pruned encodings the supremum over all ratios lies in the interval $(0.002095, 0.002096]$, and only ratios in $(21, 23)$ come within 0.002095 (Theorem 6.6); with the paper’s encodings it lies in $(0.002059, 0.002061]$, approached only for ratios in $(20.8, 22)$ (Theorem 6.7). The theorems reach every saving below the two suprema: the upstream program tests $D^{18} \leq N$ to decide whether its recursion applies, which excludes the optimal ratios, and we replace the test by one with a rational exponent (Section 7.2). So the method, with this identity and these ingredients, ends at a saving of about 0.0020953 for Exact Triangle, that is, at the exponent $2 - 0.0020953/2$ for 3SUM. Section 9 explains why no identity of the same kind does better.

Results. Table 1.1 lists the exponents. The column “paper” is [1]; each of the next columns adds one ingredient to the previous one.

All four columns of new results hold without any hypothesis. The first of them uses none of the ingredients: it is the algorithm and the analysis of the paper at better parameters. The first three use the existing encoder, which is the paper’s; the last uses the pruned encoder of Theorem 4.6. Theorems 7.4 and 7.5 give every saving below the two suprema of Theorems 6.7 and 6.6; the decimal exponents are their values at one ratio each.

	paper	Thm. 7.2	Thm. 7.3	Thm. 7.4	Thm. 7.5
added		parameters	exact count	middle size	pruned enc.
saving δ	0.00175	0.00184	0.00196	0.002059	0.002095
Exact Triangle	2.9983	2.99816	2.99804	2.99795	2.99791
3SUM	1.9992	1.99908	1.99902	1.99898	1.99896
(min, +)-product, APSP	2.99942	2.99939	2.99935	2.99932	2.99931
all-edges route				2.99795	2.99791
status	[2]	checked	checked	checked	checked

TABLE 1.1. Exponents a of the running times $O(n^a)$ of deterministic word-RAM programs. The exponents of the column “paper” are those printed in [1] and proved in [2]. The all-edges route (Theorem 7.10) is carried out for the two solvers of the last two columns.

Formalization. The results are formalized in Lean 4 [4] with Mathlib [9], on top of the formalization [2] of the original paper; the development and this paper are in the repository <https://github.com/qawbecrdtey/ImprovedExponents>. Every time a result is machine-checked, the text names the Lean declarations in a typewriter face, as in `sum_beta_le_exp`. The heading of a numbered statement shows its status.

- *No marker*: the statement is machine-checked.
- “not formalized”: the statement is proved here on paper and is not machine-checked. The marker never means “unproved”. It occurs only in Section 9, for the mixtures of shapes, the hypothetical shapes, the experiments and a remark on ω ; every numbered statement outside Section 9 is machine-checked, and so is the comparison of the shapes in Section 9.

Section 8 says what has to be trusted and how the development is checked.

Organization. Section 2 recalls the framework of [1]. Sections 3, 4 and 5 have the three ingredients, Section 6 the optimal parameters and Section 7 the theorems for 3SUM, Exact Triangle, the (min, +)-product and APSP, together with the all-edges route to APSP. Section 8 describes the formalization and Section 9 the limits of the method.

2. THE FRAMEWORK OF ALMAN AND VASSILEVSKA WILLIAMS

We recall the parts of [1] that are used. Numbers of lemmas and theorems “of the paper” are those of [1]. All logarithms are natural unless a base is written, $H(x) = -x \ln x - (1-x) \ln(1-x)$ is the entropy function, and $\psi(x) = x \ln x$ with $\psi(0) = 0$.

2.1. Problems and the machine. *Exact Triangle* asks whether a weighted tripartite graph with n vertices per part has a triangle of weight zero. *3SUM* asks whether n integers contain three with sum zero. The (min, +)-*product* of two $n \times n$ integer matrices A, B is $C_{ij} = \min_k (A_{ik} + B_{kj})$, and *APSP* asks for all distances in a directed graph with integer weights and no negative cycle.

The *thin product* is the following task. Given $X \in \mathbb{Z}^{N \times D}$, $Y \in \mathbb{Z}^{D \times N}$ and a set W of *wanted* positions of the $N \times N$ matrix XY , compute $(XY)[I, J]$ for all $(I, J) \in W$. The number D is the *inner dimension*. `Lop-AE-SparseTri( $n, D$ )` (Definition 13 of the paper) is the special case that Exact Triangle reduces to: a tripartite graph with outer parts of n vertices and a middle part of at most D vertices, and a set W of *query pairs* of outer vertices; decide for each pair whether it has a common neighbor in the middle part.

Running times are those of programs of a word RAM, in the sense of the formalization [2] of the paper. For a problem Q and a rational number r , the statement **Q.SolvedInTime** r says: for every κ there are a program, a slope b and a bound $T(n) = O(n^r)$ such that the program is correct, within $T(n)$ steps, on all instances of size n whose numbers have absolute value at most n^κ , at every word size $\geq b(\lfloor \log_2 n \rfloor + 1)$. The formalization proves Theorem 22 of the paper in this form, with $r = 1.9992$ for 3SUM and $r = 2.99942$ for the $(\min, +)$ -product and for APSP.

2.2. The recursion. The recursion of Section 2 of the paper applies Schönhage's identity [7] for $\langle 3, 1, 3 \rangle \oplus \langle 1, 4, 1 \rangle$. The identity has ten terms. Nine of them feed one outer output each, and the tenth, P_0 , is shared. It has seven left variables: three outer ones and four inner ones.

The identity is applied on L levels, m of which are inner. We write

$$D = 4^m, \quad N_0 = 3^{L-m}, \quad K = \binom{L}{m}, \quad M = KN_0^2 = \binom{L}{m} 9^{L-m}, \quad c = \frac{L}{m} \geq 10.$$

The matrix XY is cut into *tiles* of side $\sqrt{K}N_0$ (at most $4N^2/M$ of them), each with M output strings, and the rows of X into *bands* of $\sqrt{K}N_0$ rows.

A *leaf* is a string of L terms. Its *order* is m minus its number of symbols P_0 . For a set U of output strings, $\text{Leaves}(U)$ is the set of leaves at which the recursion multiplies for some string of U ; these leaves have order between 0 and m . Two counts govern the cost:

$$\alpha_d = \binom{m}{d} 9^d, \quad \beta_d = \binom{L}{m-d} 9^{L-m+d} \quad (0 \leq d \leq m).$$

One output string is fed by α_d leaves of order d , and a tile has β_d leaves of order d ; $\beta_0 = M$. Lemma 11 of the paper is the bound

$$|\text{Leaves}(U)| \leq \sum_{d=0}^m \min(|U| \alpha_d, \beta_d) \quad (2.1)$$

for a set U of output strings of one tile, followed by an estimate of the right-hand side.

2.3. Boxes and queries. Section 4 of the paper (Theorem 30) turns the recursion into a data structure with a switching order $0 \leq t \leq m$. The leaves of order at least t are covered by precomputed *boxes*, at most

$$(m+1) \sum_{d=t}^m \beta_d \quad (2.2)$$

per tile (Lemma 29 of the paper). A query for one entry sums over the leaves of order below t and costs $O(L \sum_{d \leq t} \alpha_d)$. The encodings $\Phi_\tau(a)$ of the input at all 10^L leaves τ are computed for every band by Yates' algorithm [13]. The preprocessing costs a constant times the expression (8) of the paper,

$$Lm \frac{\rho^t}{1-\rho} N^2 + \frac{N \cdot 10^L}{\sqrt{K}N_0}, \quad \rho = \frac{9m}{L-m+1}, \quad (2.3)$$

because the paper bounds $\sum_{d \geq t} \beta_d \leq M\rho^t/(1-\rho)$; the second summand is the cost of the encodings.

Fix real numbers $c > 10$ and $0 < \theta < 0.9$ and let $L = \lceil cm \rceil$, $t = \lceil \theta m \rceil$. Then $\rho < \rho_c := 9/(c-1) < 1$. Put

$$\gamma_{\text{pap}}(c, \theta) = \frac{\theta \ln(1/\rho_c)}{\ln 4}, \quad q(\theta) = \frac{H(\theta) + \theta \ln 9}{\ln 4}. \quad (2.4)$$

The first summand of (2.3) is $O(N^2 \log^2 D / D^{\gamma_{\text{pap}}})$ and a query costs $O(D^{q(\theta)} \log D)$. The second summand is at most N^2 / D^γ when the instance is *thin*: $D \leq N^\varepsilon$ with

$$\varepsilon < R_c(\gamma) := \frac{\ln 4}{A_{\text{full}}(c) + \gamma \ln 4}, \quad A_{\text{full}}(c) = c \ln 10 - \frac{c}{2} H\left(\frac{1}{c}\right) - (c-1) \ln 3. \quad (2.5)$$

This is equation (11) of the paper; $A_{\text{full}}(c)$ is the exponential rate of $10^L / (\sqrt{K} N_0)$ per inner level. Corollary 31 of the paper is the resulting data structure, and Corollary 32 the offline form: for $|W| \leq N^2 / D^\kappa$ wanted entries the thin product takes time

$$O(N^2 \log^2 D (D^{-\gamma} + D^{q(\theta)-\kappa})), \quad \gamma = \gamma_{\text{pap}}(c, \theta). \quad (2.6)$$

Corollary 26 of the paper takes $c = 21$, $\theta = 1/9$, $\kappa = 1/2$ and $\varepsilon = 1/18$, which gives $\gamma = 0.063$.

2.4. From the thin product to Exact Triangle. Theorem 17 of the paper reduces Exact Triangle on n vertices per part with weights of absolute value at most n^ν to Lop-AE-SparseTri. Let $16 \leq D' \leq n$ and let $1 \leq g' \leq \sqrt{D'}$ be an integer. The weights are hashed modulo a prime $p \in [\sqrt{D'}/2, \sqrt{D'})$ that is chosen to minimize the number of false positives. The reduction produces at most $4ng'$ instances of Lop-AE-SparseTri(n, D') with at most $n^2/\sqrt{D'}$ query pairs each, and takes

$$O\left(\frac{\nu n^3 \log n}{g'} + n^{\omega+o(1)} D'^{3/2} + n^2 D' g'\right) \quad (2.7)$$

additional time. The three terms are the scans for witnesses, the choice of the prime and the construction of the instances. The middle part of an instance is $C_k \times \mathbb{Z}_p$ for a piece C_k of one part with $|C_k| \leq \lceil s/g' \rceil$, $s = \lfloor \sqrt{D'} \rfloor$. We write D' and g' for the parameters of the reduction and keep D for the inner dimension of the thin products; in the paper the two agree.

With $D' = n^{1/18}$ and $g' = D'^{\gamma/2}$, $\gamma = 0.063$, the calls and the scans balance (Remark 20 of the paper), and Exact Triangle takes time $n^{3-0.00175}$ up to a logarithmic factor (Theorem 19 of the paper), since $0.063/36 = 0.00175$.

2.5. From Exact Triangle to 3SUM and APSP. Theorem 21 of the paper recalls two known deterministic reductions. 3SUM on n numbers reduces to $n^{1/2+o(1)}$ instances of Exact Triangle on $n^{1/2+o(1)}$ vertices per part [3, 12]. The $(\min, +)$ -product of $n \times n$ matrices with entries up to U takes time $O(n^2 T(n^{1/3}) \log^2 U)$ if Exact Triangle on s vertices per part takes time $T(s)$, and APSP follows with one more logarithm [11, 10, 12]. So if Exact Triangle is solved in time $n^{3-\delta}$, then 3SUM is solved in time n^a for every $a > 2 - \delta/2$, and the $(\min, +)$ -product and APSP in time n^a for every $a > 3 - \delta/3$: 3SUM keeps half of the saving and APSP a third. With $\delta = 0.00175$ this is Theorem 22 of the paper: 3SUM in time $n^{1.999125+o(1)}$, printed as $O(n^{1.9992})$, and APSP in time $O(n^{2.99942})$.

3. THE EXACT COUNT OF THE LEAVES

The paper bounds the tail $\sum_{d \geq t} \beta_d$ of the leaf counts by a geometric series. This section bounds it by its true exponential rate. The program does not change; only its analysis does.

3.1. The rate function.

Definition 3.1. For $c > 1$ and $0 \leq \delta \leq 1$ let

$$g_2(c, \delta) = \psi(c-1) - \psi(c-1+\delta) - \psi(1-\delta) + \delta \ln 9 = c H\left(\frac{1-\delta}{c}\right) - c H\left(\frac{1}{c}\right) + \delta \ln 9,$$

and $\gamma_X(c, \theta) = -g_2(c, \theta) / \ln 4$.

The two expressions agree because $cH(x/c) = c \ln c - \psi(x) - \psi(c-x)$ for $0 \leq x \leq c$. In Lean the functions are `g2` and `gammaX`.

Lemma 3.2. *Let $c \geq 10$.*

- (i) $\partial g_2 / \partial \delta = \ln(9(1-\delta)/(c-1+\delta)) \leq 0$ for $0 < \delta < 1$. So $g_2(c, \cdot)$ is nonincreasing on $[0, 1]$, and strictly decreasing if $c > 10$.
- (ii) $\partial g_2 / \partial c = \ln((c-1)/(c-1+\delta)) \leq 0$. So $g_2(\cdot, \delta)$ is nonincreasing on $[1, \infty)$.
- (iii) $g_2(c, 0) = 0$, and $g_2(c, \delta) < 0$ for $c > 10$ and $0 < \delta \leq 1$.
- (iv) $g_2(c, \delta) \leq \delta \ln \rho_c$ with $\rho_c = 9/(c-1)$. Hence $\gamma_{\text{pap}}(c, \theta) \leq \gamma_X(c, \theta)$ for $0 \leq \theta \leq 1$, and for $c > 10$ the inequality is strict unless $\theta = 0$.

Proof. (i) and (ii) follow from $\psi'(x) = \ln x + 1$. The sign in (i) is that of $9(1-\delta) - (c-1+\delta) = 10 - c - 10\delta$. (iii) follows from (i). For (iv), the second derivative of $g_2(c, \cdot)$ is $-1/(c-1+\delta) - 1/(1-\delta) < 0$, so the function is strictly concave and lies below its tangent at 0, which is $\delta \mapsto \delta \ln \rho_c$. $\square$

The parts of the lemma are `hasDerivAt_g2_right`, `g2_antitone0n_right`, `g2_strictAnti0n_right`, `hasDerivAt_g2_left`, `g2_antitone0n_left`, `g2_neg` and `gammaOf_le_gammaX`; the strict inequalities of (iv), $g_2(c, \delta) < \delta \ln \rho_c$ for $\delta > 0$ and $\gamma_{\text{pap}}(c, \theta) < \gamma_X(c, \theta)$ for $\theta > 0$ and $c > 10$, are `g2_lt_mul_log_rhoC` and `gammaOf_lt_gammaX`.

3.2. The tail.

Theorem 3.3 (Exact tail). *Let $m \geq 1$, $L \geq 10m$ and $0 \leq t \leq m$. Then*

$$\sum_{d=t}^m \beta_d \leq (m+1)(L+1) M e^{m g_2(L/m, t/m)}.$$

Proof. First,

$$e^{m g_2(L/m, t/m)} = \frac{9^t m^m (L-m)^{L-m}}{(m-t)^{m-t} (L-m+t)^{L-m+t}}, \quad (3.1)$$

with $0^0 = 1$. Indeed $m \psi(x/m) = \psi(x) - x \ln m$, so $m g_2(L/m, t/m) = \psi(L-m) + \psi(m) - \psi(L-m+t) - \psi(m-t) + t \ln 9$, the multiples of $\ln m$ cancelling.

So it suffices to prove, for $t \leq d \leq m$, the inequality between integers

$$\beta_d (m-t)^{m-t} (L-m+t)^{L-m+t} \leq (L+1) M 9^t m^m (L-m)^{L-m}, \quad (3.2)$$

and to sum it over the at most $m+1$ values of d . It follows from three facts.

- (i) For $j \leq j_0 \leq L/10$, $\binom{L}{j} 9^{j_0-j} \leq \binom{L}{j_0}$: the ratio of consecutive binomial coefficients is $(L-j)/(j+1) \geq 9$ for $j < L/10$.
- (ii) $\binom{L}{j_0} j_0^{j_0} (L-j_0)^{L-j_0} \leq L^L$: the left-hand side is one term of the binomial expansion of $(j_0 + (L-j_0))^L$.
- (iii) $L^L \leq (L+1) \binom{L}{m} m^m (L-m)^{L-m}$: the expansion of $(m + (L-m))^L$ has $L+1$ terms, and the term with index m is the largest.

Apply (i) with $j = m-d$ and $j_0 = m-t$, which is at most $L/10$: $\beta_d = \binom{L}{m-d} 9^{d-t} \cdot 9^{L-m+t} \leq \binom{L}{m-t} 9^{L-m+t}$. By (ii) with the same j_0 and then (iii), the left-hand side of (3.2) is at most $9^{L-m+t} L^L \leq 9^{L-m+t} (L+1) \binom{L}{m} m^m (L-m)^{L-m}$, which is the right-hand side because $M = \binom{L}{m} 9^{L-m}$. $\square$

In Lean: `exp_mul_g2_div` is (3.1), `choose_mul_nine_pow_le` is fact (i), `beta_mul_le` is (3.2), and `sum_beta_le_exp` is the theorem.

Corollary 3.4. *Let $c > 10$, $0 \leq \theta \leq 1$, $m \geq 1$, $L = \lceil cm \rceil$, $t = \lceil \theta m \rceil$ and $D = 4^m$. Then*

$$\sum_{d=t}^m \beta_d \leq (m+1)(L+1) M D^{-\gamma_X(c,\theta)}.$$

Proof. $L/m \geq c$ and $\theta \leq t/m \leq 1$, so $g_2(L/m, t/m) \leq g_2(c, \theta)$ by Lemma 3.2(i),(ii), and $e^{m g_2(c,\theta)} = D^{-\gamma_X(c,\theta)}$. $\square$

In Lean: `sum_beta_le_rpow`.

3.3. The offline routine with the exact exponent. By (2.2) and Corollary 3.4, the boxes of all tiles number at most $4(m+1)^2(L+1)N^2 D^{-\gamma_X(c,\theta)}$. For $\gamma < \gamma_X(c, \theta)$ the factor $(m+1)(L+1)$ is absorbed by $D^{\gamma_X - \gamma}$. So the first summand of (2.3) can be replaced by $O(N^2 \log^2 D / D^\gamma)$ for every $\gamma < \gamma_X(c, \theta)$; the expression with the exact tail in place of the geometric bound is `cost8X`.

Theorem 3.5 (The offline routine). *Let $c = a/b > 10$ and $\theta = p/q \in (0, 0.9)$ be rational, let $0 < \gamma < \gamma_X(c, \theta)$ and $\varepsilon < R_c(\gamma)$. For every large enough threshold m_0 , the offline routine of Section 4.4 of the paper at the parameters (c, θ, m_0) , as a program, runs in at most a constant times*

$$\frac{N^2(\log D + 1)^2}{D^\gamma} + w D^{q(\theta)}(\log D + 1)$$

steps on every instance with $1 \leq D \leq N^\varepsilon$ and w wanted positions.

The program is `program31` of [2], unchanged. Its time is bounded upstream by a constant times any pair of bounds that dominate the two costs of Theorem 30; the proof is repeated against `cost8X` (`exists_tPreCore_leX`, `exists_tOffline32_leX`), and the corollary above supplies the domination (`first_termX`, `costsX`). The theorem is `offlineWithinX`. The statement that a procedure of the program solves the thin product within these bounds on all instances with $D \leq N^\varepsilon$ is `thinClaim_gammaX`; it has two more hypotheses, which come from the regime $\text{test } D^{18} \leq N$ in the program text and are discussed in Section 7.

3.4. The form of Section 2. The same count improves Lemma 11 of the paper directly. The theorems of Section 7 use the form of Section 4 above and do not need the following statement.

Proposition 3.6 (Lemma 11 with the exact count). *Let $c \geq 1$, $\kappa \in \mathbb{R}$, $m \geq 1$ and $L \geq \max(10m, cm)$. Let W be a set of wanted positions of an $N \times N$ matrix with $|W| \leq N^2/D^\kappa$, and for each tile T let W_T be the set of output strings of T that are wanted. Then*

$$\sum_T |\text{Leaves}(W_T)| \leq \sum_{d=0}^m \min(|W| \alpha_d, \tau \beta_d) \leq 4(m+1)(L+1) N'^2 e^{m \Gamma_c(\kappa)},$$

where τ is the number of tiles, N' is the size of the matrix padded to a multiple of the side of a tile, and

$$\Gamma_c(\kappa) = \sup_{0 \leq \delta \leq 1} \min(f_1(\delta), g_2(c, \delta)), \quad f_1(\delta) = -\kappa \ln 4 + H(\delta) + \delta \ln 9 = (q(\delta) - \kappa) \ln 4.$$

If the side of a tile is at most N , then $N' \leq 2N$.

So the number of leaves is at most $16(m+1)(L+1)N^2 D^{-\gamma_c(\kappa)}$ with $\gamma_c(\kappa) = -\Gamma_c(\kappa)/\ln 4$.

Proof. By (2.1), $|\text{Leaves}(W_T)| \leq \sum_d \min(|W_T| \alpha_d, \beta_d)$. A sum of minima is at most the minimum of the sums, and $\sum_T |W_T| \leq |W|$; this is the first inequality. Fix d and put $\delta = d/m$. First, $\alpha_d = \binom{m}{d} 9^d \leq e^{m(H(\delta) + \delta \ln 9)}$, so $|W| \alpha_d \leq N^2 e^{m f_1(\delta)} \leq N'^2 e^{m f_1(\delta)}$. Second, $\tau M \leq 4N'^2$, and by the inequality (3.2) with $t = d$ and by (3.1), $\beta_d/M \leq (L+1)e^{m g_2(L/m, \delta)} \leq (L+1)e^{m g_2(c, \delta)}$,

where the last step is Lemma 3.2(ii). So each of the $m + 1$ values of d contributes at most $4(L + 1)N'^2 e^{m \min(f_1(\delta), g_2(c, \delta))}$. $\square$

In Lean (file `ExactCount/Leaves.lean`): f_1 is `f1` and $\Gamma_c(\kappa)$ is `GammaMM`; the first inequality is `sum_card_Leaves_le_sum_min`, the second is `sum_card_Leaves_le_exp_padN`, and the form with $16N^2$ is `sum_card_Leaves_le_exp`.

Remark 3.7. f_1 increases strictly on $[0, 0.9]$ and $g_2(c, \cdot)$ is nonincreasing, so the supremum is taken where they cross: for every $\delta_0 \in [0, 0.9]$, $\min(f_1, g_2)(\delta_0) \leq \Gamma_c(\kappa) \leq \max(f_1, g_2)(\delta_0)$ (`min_le_GammaMM`, `GammaMM_le_max`). For $c > 10$ and $0 < \kappa < \log_4 10$ they do cross, and $\Gamma_c(\kappa) < 0$ (`exists_crossing_f1_g2`). At the crossing $\gamma_X(c, \delta) = \kappa - q(\delta)$, which is also where the two terms of (2.6) balance when γ is replaced by γ_X . The two forms have the same exponent.

4. PRUNED ENCODINGS

The second summand of (2.3) pays for the encodings of the input at all 10^L leaves. Most of them are never read. This section computes only those that are. First the mathematics: the correctness of the pruned recursion, its number of operations, and the resulting cost of the method (Theorem 4.2, Proposition 4.5). Then the program: the pruned encoder is written in the language of [2], the procedures of the paper's data structure are re-verified under the weaker contract that the encodings are right only at the leaves that are read, and the running time of the new solver is proved on the machine (Theorem 4.6). Everything is machine-checked.

4.1. What is read and what is nonzero. Let a be the array of a band. It is indexed by the *left strings*: strings $u = u_1 \cdots u_L$ of left variables. For a leaf $\tau = \tau_1 \cdots \tau_L$ the encoding is

$$\Phi_\tau(a) = \sum_u a[u] \prod_{\ell=1}^L \varphi_{\tau_\ell}(u_\ell),$$

where φ_τ is the linear form of the term τ in the left variables. Each φ_τ has at most three nonzero coefficients. Two facts hold in the recursion.

- Only leaves with at most m symbols P_0 are read: the leaves of `Leaves(U)` have order at least 0.
- $a[u] = 0$ unless u has exactly m inner variables.

Yates' algorithm computes the encodings level by level. Stage k holds

$$A_k[\tau_1 \cdots \tau_k; u_{k+1} \cdots u_L] = \sum_{u_1, \dots, u_k} a[u] \prod_{\ell \leq k} \varphi_{\tau_\ell}(u_\ell),$$

so $A_0 = a$ and $A_L[\tau] = \Phi_\tau(a)$, and $A_{k+1}[\tau_1 \cdots \tau_{k+1}; \cdot] = \sum_v \varphi_{\tau_{k+1}}(v) A_k[\tau_1 \cdots \tau_k; v \cdot]$.

Definition 4.1. Let P_k be the set of strings of k terms with at most m symbols P_0 , and U_k the set of strings of $L - k$ left variables with at most m inner variables. The *pruned recursion* computes at stage k only the entries of A_k with index in $P_k \times U_k$, by the rule above, and treats all other entries as 0.

4.2. Correctness and cost.

Theorem 4.2 (Pruned encodings). *Let $L \geq 10m$, and let a vanish off the left strings with exactly m inner variables.*

- The pruned recursion computes $\Phi_\tau(a)$ for every leaf τ with at most m symbols P_0 .
- $|P_k| = \sum_{j \leq m} \binom{k}{j} 9^{k-j}$ and $|U_k| = \sum_{i \leq m} \binom{L-k}{i} 4^i 3^{L-k-i}$.
- $\sum_{k=0}^L |P_k| |U_k| \leq \frac{9}{2} |P_L| \leq \frac{9}{2} \cdot \frac{M}{1-\rho}$ with $\rho = 9m/(L - m + 1)$.

- (iv) Every computed entry is a sum of at most three products. So the pruned recursion makes at most $\frac{27}{2}M/(1-\rho)$ multiplications.

Proof. (i) An entry of A_k whose suffix has more than m inner variables is 0: every u in its sum has more than m inner variables, so $a[u] = 0$. An entry of A_{k+1} with prefix in P_{k+1} reads entries of A_k whose prefix is in P_k , since deleting a term does not add a symbol P_0 . So, by induction on k , the pruned arrays agree with those of Yates' algorithm at every index whose prefix has at most m symbols P_0 : such an entry is 0 if its suffix is outside U_k , and otherwise each entry it reads either lies in $P_{k-1} \times U_{k-1}$ or is 0 in both arrays.

(ii) There are nine terms other than P_0 , and four inner and three outer left variables.

(iii) Since $\binom{k+1}{j} \geq \binom{k}{j}$, we have $|P_{k+1}| \geq 9|P_k|$. By Pascal's rule,

$$\sum_{i \leq m} \binom{n+1}{i} 4^i 3^{n+1-i} = 3 \sum_{i \leq m} \binom{n}{i} 4^i 3^{n-i} + 4 \sum_{i \leq m-1} \binom{n}{i} 4^i 3^{n-i},$$

so $|U_k| \leq 7|U_{k+1}|$. With $|U_L| = 1$ this gives $|P_k||U_k| \leq (7/9)^{L-k}|P_L|$, and $\sum_k (7/9)^{L-k} \leq 9/2$. The terms of $|P_L| = \sum_{j \leq m} \binom{L}{j} 9^{L-j}$ have consecutive ratios $\binom{L}{j-1} 9 / \binom{L}{j} = 9j/(L-j+1) \leq \rho < 1$, and the term with $j = m$ is M . So $|P_L| \leq M/(1-\rho)$.

(iv) follows from (iii) and the three coefficients of a form. $\square$

In Lean (directory `PrunedEncoding`): Yates' algorithm is `yatesStage` with `yatesStage_final`; the pruned recursion is `prunedStage`; part (i) is `prunedStage_eq_yatesStage` and `prunedStage_final`; part (ii) is `card_stageSet_of_le` with `prefCount` and `sufCount`; part (iii) is `sum_card_stageSet_le`; part (iv) is `prunedMults_le_M`; the theorem as a whole is `pruned_encoding`.

Remark 4.3. $|P_L| = \sum_{d=0}^m \beta_d$: the pruned recursion computes exactly the encodings at the leaves of order at least 0. For $L \geq cm$ with $c > 10$ fixed, $1/(1-\rho) \leq (c-1)/(c-10)$.

Remark 4.4 (The work of the program). The program of Section 4.4 below follows the recursion of Theorem 4.2 with one simplification: at stage k it keeps the entries with index in $P_k \times$ (all 7^{L-k} suffixes) instead of $P_k \times U_k$, so that the suffixes are indexed densely and no compact indexing of U_k is needed. For $L \geq 10m$ the largest slice, with 7^L entries, is no larger than a tile: $7^L \leq 9^{L-m} \leq M$ (`seven_pow_le_M`). The number of computed entries, `encWorkP L m` in Lean, is

$$\sum_{k=0}^L |P_k| 7^{L-k} \leq \frac{9}{2} |P_L| \leq \frac{9}{2} \cdot \frac{M}{1-\rho},$$

from $9|P_k| \leq |P_{k+1}|$ and $\sum_i (7/9)^i \leq 9/2$ (`encWorkP_le`, `encWorkP_le_M_div`): part (iii) with 7^{L-k} in place of $|U_k|$, and the same bound. The time of the program is at most 778 times this number (`tEncP_le`).

4.3. The thinness bound. With $O(L)$ word operations per entry for the index arithmetic, a band costs $O(LM)$ instead of $O(L10^L)$, and all $N/(\sqrt{K}N_0)$ bands cost $O(LN\sqrt{K}N_0)$, since $M = KN_0^2$. This is at most N^2/D^γ up to the factor L when $\sqrt{K}N_0D^\gamma \leq N$. So the thinness condition (2.5) becomes

$$\varepsilon < R'_c(\gamma) := \frac{\ln 4}{A_{\text{pr}}(c) + \gamma \ln 4}, \quad A_{\text{pr}}(c) = \frac{c}{2} H\left(\frac{1}{c}\right) + (c-1) \ln 3, \quad (4.1)$$

where $A_{\text{pr}}(c)$ is the exponential rate of $\sqrt{K}N_0$ per inner level. Since $M \leq 10^L$, we have $A_{\text{pr}}(c) \leq A_{\text{full}}(c)$ and $R'_c(\gamma) \geq R_c(\gamma)$. In Lean the constants are `basePruned` and `baseFull`, the bound is `RcP`, and the two inequalities are `basePruned_le_baseFull` and `Rc_le_RcP`.

Proposition 4.5 (The costs with pruned encodings). *Let $c > 10$, $0 < \theta < 0.9$, $0 \leq \gamma < \gamma_X(c, \theta)$ and $0 < \varepsilon < R'_c(\gamma)$. There are C and m_0 such that the following holds for all $D \geq 2$ and N with $D \leq N^\varepsilon$ and $m = \lceil \log_4 D \rceil \geq m_0$, with $L = \lceil cm \rceil$ and $t = \lceil \theta m \rceil$.*

- (i) *The tile fits: $\sqrt{K}N_0 \leq N$.*
- (ii) *The cost of the preprocessing with pruned encodings,*

$$L m \frac{\sum_{d=t}^m \beta_d}{M} N^2 + \frac{N}{\sqrt{K}N_0} \cdot L \cdot \frac{27}{2} \cdot \frac{M}{1-\rho},$$

is at most $C N^2 \log^2 D / D^\gamma$.

- (iii) *The cost $L \sum_{d \leq t} \alpha_d$ of a query is at most $C D^{q(\theta)} \log D$.*

Proof. The first summand of (ii) is bounded as in Section 3. The second is $\frac{27}{2} N L \sqrt{K} N_0 / (1 - \rho)$. Now $\sqrt{K} N_0 \leq e^{m A_{\text{pr}}(L/m)}$, the ratio L/m tends to c , and $N \geq D^{1/\varepsilon} > 4^{(m-1)/\varepsilon}$; so $D^\gamma L \sqrt{K} N_0 \leq N$ for all large m , because $A_{\text{pr}}(c) + \gamma \ln 4 < \ln 4/\varepsilon$. This gives (i) and (ii). Part (iii) is as in the paper. $\square$

In Lean (file `PrunedEncoding/Costs.lean`): the expression of (ii) is `cost8P`, the inequality $D^\gamma L \sqrt{K} N_0 \leq N$ is `eq_10P`, and the proposition is `costsP`. It is the analogue of the statement `costsX` on which Theorem 3.5 rests, with the pruned encoder's count of operations in place of 10^L and R'_c in place of R_c .

Theorem 4.6 (The encoder program). *For all rational $c = a/b > 10$ and $\theta = p/q \in (0, 0.9)$, every $0 < \gamma < \gamma_X(c, \theta)$ and every $\varepsilon < R'_c(\gamma)$, there is a procedure, in the programming language of [2], that solves the thin product on all instances and takes at most a constant times*

$$\frac{N^2(\log D + 1)^2}{D^\gamma} + w D^{q(\theta)}(\log D + 1)$$

steps on every instance with $1 \leq D \leq N^\varepsilon$ and at most w wanted positions.

Theorem 4.6 is the conclusion of Theorem 3.5 with R'_c in place of R_c . On paper it follows from Theorem 4.2 and Proposition 4.5: replace the encoder of the offline routine by the pruned recursion, and replace the regime test of the program, which is tied to $\varepsilon \leq 1/18$, by the test with a rational exponent of Section 7.2. In Lean the statement is the proposition `PrunedEncoderClaim` (file `Pipeline/General.lean`), proved as `prunedEncoderClaim` (file `Pipeline/PrunedClaim.lean`). It is stated with the same notion, `ThinClaim`, in which Theorems 7.3 and 7.4 obtain the thin product from the existing program (`thinClaim_gammaX`, `thinClaimRS_gammaX`): a claim about the time model `lightModel` of the upstream programs, which [2] compiles to the word RAM. The rest of this section describes the program and its proof.

4.4. The encoder program. *The encoder.* Upstream's encoder is a recursive procedure `encodeBody` that descends the ten terms of the identity level by level and fills the 10^L encodings of a band. The pruned encoder (`encodePBody`, file `PrunedProgram/Encode/Recursion.lean`) is the same procedure with a budget j of symbols P_0 : a loop over the nine terms $\lambda \neq P_0$ recurses with the same budget, and the term P_0 (digit 9) is treated after the loop, only if the budget is nonzero, with budget $j - 1$. The last term is treated outside the loop because the language's `for` charges every round the same time. The slices are the dense ones of Remark 4.4. Its correctness is `encodeP_spec`: after the call with budget j , every leaf with at most j symbols P_0 holds its encoding (the postcondition `EncodePostP`), and its time is at most 778 times the work `encWorkP L j` (`tEncP_le`), which Remark 4.4 bounds by $\frac{9}{2} \cdot 778 \cdot M / (1 - \rho)$ for $j = m$.

The weaker contract. Upstream's data structure assumes that the encodings of a band are right at all 10^L leaves (a *segment*). With the pruned encoder they are right only at the leaves with at

most m symbols P_0 ; this is the predicate `SegOn` (file `PrunedProgram/SegOn.lean`). The preprocessing and the query of Theorem 30 of the paper read an encoding cell in exactly two places, both at such leaves: in the construction of the trie, at the leaf of a code of `nineStrs`, which has at most $m-t$ digits 9; and in a query, at a leaf that contributes to the output string, which has the symbol P_0 only at inner levels (`P0Levels_subset_innerSet0`, `card_P0Levels_le_of_contributes`). So upstream's own procedures for the tiles (`fillListBody`, `tileBody`, `allTilesBody`) and for a query (`queryAtBody`) are re-verified under the weaker contract, with their program text unchanged: `fillListP_meets` and `preCoreP_of_base58` (files `PrunedProgram/Tiles`, `PrunedProgram/Pre/Preprocessing.lean`), `queryCoreP_spec` and `queryAtP_of_base58` (directory `PrunedProgram/Query`). Upstream's shared stage, which builds the tables and encodes every band, becomes `sharedPBody` with the entry `sharedP_entry` (file `PrunedProgram/Shared/Stage.lean`): its two calls of the encoder are exchanged for calls of the pruned one, and nothing else changes. The invariant of the finished data structure is `DSReadyP` (file `PrunedProgram/Pre/Defs.lean`).

The program. The routines with rational parameters are `pre31PBody`, `offline32PBody` and `offline32P_meets` (directory `PrunedProgram/Wrapper`). The program is `programP`: the program `programRS` of Section 7.2 followed by seven procedures, `encodeP`, `encodeBandsP`, `sharedP`, `preCoreP`, `pre31P`, `offline32P` and `allP`; every other procedure is upstream's, with its number. The offline routine as an object of the pipeline is `offline32PRoutine`.

The cost. The running time is analyzed against `cost8P` as the running time of upstream's program is analyzed against `cost8X` in Theorem 3.5: `Within8P` and `exists_tPreCore_leP`, `CostsWithinP` and `offlineWithinP` (directory `Cost8P`) are the analogues of `exists_tPreCore_leX`, `costsX` and `offlineWithinX`. The small summands of the shared stage are bounded by $NL\sqrt{K}N_0$ directly, since `cost8P` has no term 10^L .

The pipeline. The solver of all instances is generic in the offline routine: the structure `OfflineRoutine` (file `Pipeline/ThinClaim.lean`) packages the procedure, its time, what it needs from the program and its specification, as the regime test is already packaged as `RegimeTest`. Upstream's routine is the instance `offline32Routine` and the pruned one is `offline32PRoutine`; `thinClaimRS_of_offlineWithinT` (file `Pipeline/RegimeRS.lean`) turns either into a thin-product claim at the regime $D^r \leq N^s$. For `thinClaimP_gammaX` and `prunedEncoderClaim` the fraction r/s is chosen strictly between $A_{\text{pr}}(c)/\ln 4$ and $1/\varepsilon$, which is possible because $\varepsilon < R'_c(\gamma) < \ln 4/A_{\text{pr}}(c)$; the case $\varepsilon \leq 0$ of the statement is reduced to a positive ε by `ThinClaim.mono`.

5. CHARGING THE SOLVER AT THE TRUE SIZE OF THE MIDDLE PART

In the reduction of Section 2.4 the middle part of an instance is $C_k \times \mathbb{Z}_p$ with $|C_k| \leq \lceil s/g' \rceil$ and $p \leq s$, $s = \lfloor \sqrt{D'} \rfloor$. So it has at most

$$\text{mid}(D', g') := \left\lceil \frac{s}{g'} \right\rceil s \approx \frac{D'}{g'}$$

vertices. The paper hands the instance to the solver as an instance of `Lop-AE-SparseTri`(n, D') and charges it accordingly. Handing it over as an instance of `Lop-AE-SparseTri`($n, \text{mid}(D', g')$) changes one assignment of the program and separates two roles of D' : the number of query pairs is governed by $\sqrt{D'}$, the cost of a call by D'/g' .

5.1. The reduction.

Theorem 5.1 (Theorem 17 at the true middle size). *There is a constant C with the following property. Let $T(n, D, w)$ bound the time of a solver of `Lop-AE-SparseTri`(n, D) with w query*

pairs. Then *Exact Triangle* on n vertices per part with weights of absolute value at most n^ν , $\nu \geq 1$, is solved in time at most

$$4ng' \left(T(n, \text{mid}(D', g'), \lfloor n^2/\sqrt{D'} \rfloor) + C \frac{n^2}{\sqrt{D'}} \right) + C \left(\frac{\nu n^3 \log n}{g'} + n^{\log_2 7} D'^{3/2} + n^2 D' g' \right)$$

whenever $16 \leq D' \leq n$ and $1 \leq g' \leq \sqrt{D'}$.

This is a statement about programs: the solver is a procedure of a program of the machine, and the theorem gives a program for *Exact Triangle*. The exponent $\log_2 7$ is that of Strassen's algorithm [8], which the upstream program uses for the choice of the prime; see Section 9.

In Lean the statement is `Claim17Mid`, the upstream statement `Claim.Theorem_17` with `mid(D', g')` (`midSize`) in place of D' in the first argument of T . The host is upstream's with one added assignment (`et17Sizes'`): after the prime and the size of a piece are computed, the parameter that is passed to the solver is overwritten with $\lceil s/g' \rceil s$. Its correctness and its time follow from upstream's by rewriting the arguments of the solver (`hostTime'_eq`, `obeysBound17Mid_hostTime`); the claim for programs is `claim17Mid_of_paramProcs`, and for $D' = \lfloor n^{a/b} \rfloor$, $g' = \lceil D'^{c/d} \rceil$ with rational exponents it is `hostBound_mid_rat`.

5.2. The total time.

Theorem 5.2 (Total time). *Let $\gamma, q \geq 0$ and $\varepsilon > 0$. Suppose that `Lop-AE-SparseTri`(n, D) with w query pairs is solved, for $D \leq n^\varepsilon$, in time $O(n^2(\log D + 1)^2/D^\gamma + w D^q(\log D + 1))$. Let $0 < d$ and $0 < \eta < 1/2$ be rational with $d(1 - \eta) < \varepsilon$, and let $D' = \lfloor n^d \rfloor$, $g' = \lceil D'^\eta \rceil$. Then *Exact Triangle* with weights up to n^ν is solved in time $O(\nu n^{3-\delta} \log n)$ for every $\delta \geq 0$ with*

$$\delta < d \min\left(\eta, \min(\gamma(1 - \eta), \tfrac{1}{2} - q(1 - \eta)) - \eta\right) \quad \text{and} \quad \tfrac{3}{2}d + \delta \leq 0.19. \quad (5.1)$$

Proof. Apply Theorem 5.1. The dimension of a call is $D = \text{mid}(D', g') = \Theta(D'^{1-\eta}) = \Theta(n^{d(1-\eta)})$, which is at most n^ε for large n . Up to constants and powers of $\log n$, the terms are

$$\begin{aligned} \text{calls, preprocessing:} & \quad n g' n^2 D^{-\gamma} = n^{3+d\eta-d\gamma(1-\eta)}, \\ \text{calls, queries:} & \quad n g' \frac{n^2}{\sqrt{D'}} D^q = n^{3+d\eta+dq(1-\eta)-d/2}, \\ \text{scans:} & \quad \nu n^3 / g' = \nu n^{3-d\eta}, \\ \text{prime:} & \quad n^{\log_2 7} D'^{3/2} \leq n^{2.81+3d/2}, \\ \text{construction:} & \quad n^2 D' g' = n^{2+d+d\eta}. \end{aligned}$$

The first three exponents are at most $3 - \delta$ by the first condition of (5.1), with room for the logarithms, and so is that of the term $ng'n^2/\sqrt{D'}$. The last two are at most $3 - \delta$ by the second condition. $\square$

In Lean: `explicitFrom_mid`, from `hostBound_mid_rat` and a solver of `Lop-AE-SparseTri` (`LopClaim`, obtained from a thin-product claim by `lopClaim_of_thinClaim`). The same calculation with every call charged at D' , as in the paper, is `explicitFrom_original`; its condition is $\delta < d \min(\eta, \min(\gamma, \tfrac{1}{2} - q) - \eta)$.

5.3. The saving of a tuple. It is convenient to take the dimension D of the calls as the basic parameter. Write $D \approx n^\varepsilon$ and

$$\sigma = \frac{1}{2(1 - \eta)} \in [\tfrac{1}{2}, 1), \quad \text{so that} \quad D' = D^{2\sigma}, \quad g' = D^{2\sigma-1}, \quad \sqrt{D'} = D^\sigma. \quad (5.2)$$

An instance then has inner dimension D and at most n^2/D^σ query pairs. With the thin product of Section 3 at the parameters (c, θ) the exponent of a call is $\min(\gamma_X(c, \theta), \sigma - q(\theta))$, by (2.6) with $\kappa = \sigma$, and the first condition of (5.1) reads $\delta < s(c, \theta, \sigma, \varepsilon)$ with $d(1 - \eta)$ in the place of ε , where:

Definition 5.3. The *saving* of the tuple $(c, \theta, \sigma, \varepsilon)$ is

$$s(c, \theta, \sigma, \varepsilon) = \varepsilon \min\left(2\sigma - 1, \min(\gamma_X(c, \theta), \sigma - q(\theta)) - (2\sigma - 1)\right).$$

In Lean: `tupleSaving`, with `effGamma` for the inner minimum. The first argument of the outer minimum is the saving of the scans, the second that of the calls. In the paper every call is charged at the dimension D' ; relative to D' the density of the query pairs is $\kappa = 1/2$. Relative to the true dimension D it is $\sigma > 1/2$ as soon as $g' > 1$, and the thin product profits from both: a smaller inner dimension and, relative to it, fewer wanted entries.

Remark 5.4 (A larger hash prime). The same algorithm can be described without g' . Hash modulo a prime $p \approx D^\sigma$ with $\sigma \geq 1/2$, and cut each part into pieces of $D^{1-\sigma}$ vertices, so that a middle part $C_k \times \mathbb{Z}_p$ has D vertices. There are $O(nD^{2\sigma-1})$ instances with at most n^2/D^σ query pairs each, and the scans cost $\nu n^3 \log n / D^{2\sigma-1}$. This is the reduction above with $D' = D^{2\sigma}$ and $g' = D^{2\sigma-1}$: a larger prime and no further splitting. The balance of Remark 20 of the paper, $\eta = \gamma/2$, becomes the fixed point $\sigma = 1/2 + \gamma/4$ of Section 6.

6. THE OPTIMAL PARAMETERS

The method has four real parameters: the ratio $c > 10$, the switching ratio $0 < \theta < 0.9$, the size $1/2 \leq \sigma < 1$ of the prime, and the exponent $\varepsilon > 0$ of the inner dimension. This section finds the supremum of the saving of Definition 5.3 over the parameters that the method allows.

6.1. Definitions. Let $A > 0$ be a *thinness constant* and

$$R(\gamma) = \frac{\ln 4}{A + \gamma \ln 4}, \quad F(\gamma) = \frac{R(\gamma) \gamma}{2}.$$

The paper's encodings have $A = A_{\text{full}}(c)$, so that $R = R_c$ of (2.5), and pruned encodings have $A = A_{\text{pr}}(c)$, so that $R = R'_c$ of (4.1). A tuple $(c, \theta, \sigma, \varepsilon)$ is *admissible* for A if $c > 10$, $0 < \theta < 0.9$, $1/2 \leq \sigma < 1$ and $0 < \varepsilon < R(\gamma_X(c, \theta))$. Put

$$\gamma_Q(\theta) = \frac{2 - 4q(\theta)}{3}, \quad \Gamma(c) = \sup_{0 < \theta < 0.9} \min(\gamma_X(c, \theta), \gamma_Q(\theta)).$$

In Lean: `thinR`, `savingF`, `AdmissibleX`, `gammaQ`, `Gam`.

6.2. The crossing point.

Lemma 6.1. *Let $c > 10$. On $[0, 0.9]$ the function $\gamma_X(c, \cdot)$ is continuous and strictly increasing with $\gamma_X(c, 0) = 0$, and γ_Q is continuous and strictly decreasing with $\gamma_Q(0) = 2/3$ and $\gamma_Q(0.9) < 0$. They cross at exactly one point $\theta^*(c) \in (0, 0.9)$, and $\Gamma(c)$ is their common value there. Moreover $0 < \Gamma(c) < 2/3$, and*

$$\min(\gamma_X(c, \theta), \gamma_Q(\theta)) \leq \Gamma(c) \leq \max(\gamma_X(c, \theta), \gamma_Q(\theta)) \quad (0 < \theta < 0.9).$$

Γ is nondecreasing on $(10, \infty)$.

Proof. $\gamma_X(c, \cdot)$ increases strictly by Lemma 3.2(i). The function q increases strictly on $[0, 0.9]$, since $q'(\theta) \ln 4 = \ln(9(1-\theta)/\theta) > 0$ for $\theta < 0.9$, and $q(0.9) = \log_4 10 > 1$. The intermediate value theorem gives the crossing, and monotonicity gives uniqueness and the two inequalities. The last

statement holds because γ_X is nondecreasing in c (Lemma 3.2(ii)), so that $\min(\gamma_X(c, \theta), \gamma_Q(\theta))$ is. $\square$

In Lean: `existsUnique_crossing`, `Gam_eq_of_crossing`, `Gam_pos`, `Gam_lt_two_thirds`, `Gam_ge_min`, `Gam_le_max`, `Gam_monotoneOn`. The two inequalities are what the numerical bounds below rest on: every θ gives a lower and an upper bound for $\Gamma(c)$.

6.3. The supremum for a fixed ratio.

Theorem 6.2 (Supremum of the saving). *Let $c > 10$ and $A > 0$. Over the tuples $(c, \theta, \sigma, \varepsilon)$ that are admissible for A ,*

$$\sup s(c, \theta, \sigma, \varepsilon) = S(c) := F(\Gamma(c)) = \frac{R(\Gamma(c)) \Gamma(c)}{2},$$

and the supremum is not attained. It is approached at $\theta = \theta^(c)$, $\sigma = 1/2 + \Gamma(c)/4$ and $\varepsilon \uparrow R(\Gamma(c))$.*

Proof. R is positive and strictly decreasing on $[0, \infty)$, and $F(\gamma) = \frac{1}{2}(1 - A/(A + \gamma \ln 4))$ is strictly increasing there.

Upper bound. Let the tuple be admissible, and put $a = \gamma_X(c, \theta)$, $u = 2\sigma - 1 \in [0, 1)$ and $v = \min(u, \min(a, \sigma - q(\theta)) - u)$, so that the saving is εv . If $v \leq 0$ the saving is at most $0 < F(\Gamma(c))$. Let $v > 0$. Then $v \leq u$ and $v \leq a - u$, so $v \leq a/2$. Also $v \leq \sigma - q(\theta) - u = (1 - u)/2 - q(\theta) \leq (1 - v)/2 - q(\theta)$, so $v \leq (1 - 2q(\theta))/3 = \gamma_Q(\theta)/2$. Hence $v \leq \min(a, \gamma_Q(\theta))/2 \leq \Gamma(c)/2$ by Lemma 6.1. Since $\varepsilon < R(a)$ and $v > 0$, the saving is less than $R(a)v$. If $a \leq \Gamma(c)$, then $R(a)v \leq R(a)a/2 = F(a) \leq F(\Gamma(c))$. If $a > \Gamma(c)$, then $R(a)v \leq R(\Gamma(c))\Gamma(c)/2$.

Approach. At $\theta = \theta^*(c)$ we have $\gamma_X(c, \theta) = \gamma_Q(\theta) = \Gamma$, that is, $q(\theta) = 1/2 - 3\Gamma/4$. With $\sigma = 1/2 + \Gamma/4$ we get $u = \Gamma/2$ and $\sigma - q(\theta) = \Gamma$, so $v = \Gamma/2$ and the saving is $\varepsilon\Gamma/2$, for every $0 < \varepsilon < R(\Gamma)$. Here $\sigma < 1$ because $\Gamma < 2/3$. $\square$

In Lean: `tupleSaving_lt_savingF`, `tupleSaving_crossing`, `exists_tupleSaving_gt`, `isLUB_tupleSaving` and `savingF_notMem_tupleSaving`.

Programs can only use rational parameters. Nothing is lost: the saving is continuous in the four parameters, admissibility is an open condition when A depends continuously on c , and both A_{full} and A_{pr} do.

Proposition 6.3. *Let A be A_{full} or A_{pr} , let $c > 10$ and $s < F_{A(c)}(\Gamma(c))$. For every $c_1 > c$ there are rational numbers $c', \theta', \sigma', \varepsilon'$ with $c < c' < c_1$ and $\sigma' > 1/2$ such that $(c', \theta', \sigma', \varepsilon')$ is admissible for $A(c')$ and $s(c', \theta', \sigma', \varepsilon') > s$.*

In Lean: `exists_rat_tuple_baseFull` and `exists_rat_tuple_basePruned`, from `continuous_tupleSaving`.

If every call is charged at D' , as in the paper, the condition of Theorem 5.2 is $\delta < d \min(\eta, \min(\gamma, \frac{1}{2} - q) - \eta)$ with $d < \varepsilon$. The constraint from the queries is then $\gamma \leq \frac{1}{2} - q$ instead of $\gamma \leq \sigma - q$ with $\sigma = \frac{1}{2} + \frac{\gamma}{4}$, which is $\gamma \leq \gamma_Q$. The following proposition says what this costs.

Proposition 6.4 (The paper's accounting). *Let $c > 10$ and $A > 0$. Put $\Gamma_{1/2}(c) = \sup_{0 < \theta < 0.9} \min(\gamma_X(c, \theta), \frac{1}{2} - q(\theta))$, and call $(\theta, \gamma, \varepsilon, \eta)$ admissible for A if $0 < \theta < 0.9$, $0 < \gamma < \gamma_X(c, \theta)$, $0 < \varepsilon < R(\gamma)$ and $0 < \eta < \frac{1}{2}$. Then $\gamma_X(c, \cdot)$ and $\frac{1}{2} - q$ cross in $(0, 0.9)$, $\Gamma_{1/2}(c)$ is their value at the crossing, and*

$$\sup \varepsilon \min(\eta, \min(\gamma, \frac{1}{2} - q(\theta)) - \eta) = F(\Gamma_{1/2}(c)) < F(\Gamma(c)),$$

the supremum over the admissible tuples; moreover $\Gamma_{1/2}(c) < \Gamma(c)$.

The admissible tuples are exactly the hypotheses of Theorem 7.3's general form (`allSolved_exact`), with d absorbed into ε . The supremum is approached at the crossing with $\eta = \gamma/2$. In Lean (file `Optimum/PaperAccounting.lean`): `GamHalf`, `exists_crossing_half`, `GamHalf_eq_of_crossing`, `GamHalf_lt_Gam`; the saving is `paperSaving`, the admissible tuples are `AdmissibleHalf`, the supremum is `isLUB_paperSaving`, and the strict inequality between the two suprema is `savingF_GamHalf_lt`.

6.4. Numerical values and the optimum over the ratio. Both thinness constants are nondecreasing in c on $[10, \infty)$: the derivatives are $A'_{\text{full}}(c) = \ln \frac{10}{3} + \frac{1}{2} \ln(1 - \frac{1}{c})$ and $A'_{\text{pr}}(c) = \ln 3 - \frac{1}{2} \ln(1 - \frac{1}{c})$ (`baseFull_monotoneOn`, `basePruned_monotoneOn`). So two effects compete in $S(c) = F_{A(c)}(\Gamma(c))$: Γ grows with c , and R shrinks.

Numerically, with pruned encodings S is largest at $c \approx 21.92$, where $S \approx 0.0020953$, $\Gamma \approx 0.0759$, $\sigma \approx 0.519$, $\varepsilon \approx 0.0552$ and $\theta^* \approx 0.116$. With the paper's encodings it is largest at $c \approx 21.38$, where $S \approx 0.0020599$.

These values are computed in floating point. The following are certified.

Proposition 6.5 (Values at three ratios). (i) *At $c = 107/5$ with the paper's encodings:*
 $24.8438 \leq A_{\text{full}}(c) \leq 24.84381$, $0.0741364 \leq \Gamma(c) \leq 0.0741372$ and $0.002059 < S(c) < 0.00206$.
(ii) *At $c = 108/5$ with the paper's encodings:* $A_{\text{full}}(c) \geq 18 \ln 4$, $0.074835 \leq \Gamma(c) \leq 0.074837$,
 $0.05504 < R_c(\Gamma(c)) < 0.05505$ and $0.00205 < S(c) < 0.00206$.
(iii) *At $c = 22$ with pruned encodings:* $0.0762 < \Gamma(c) < 0.0763$, $0.05498 < R'_c(\Gamma(c)) < 0.05499$ and $0.002095 < S(c) < 0.002096$.

In Lean: `baseFull_107_5_mem`, `Gam_107_5_mem`, `savingF_full_107_5_gt`, `savingF_full_107_5_lt`; `baseFull_ge_18`, `Gam_108_5_mem`, `Rc_108_5_bounds`, `savingF_full_gt`, `savingF_full_lt`; `Gam_22_bounds`, `thinR_pruned_bounds`, `savingF_pruned_gt`, `savingF_pruned_lt`.

Theorem 6.6 (The optimum over the ratio). *Let $S(c) = F_{A_{\text{pr}}(c)}(\Gamma(c))$ be the supremum of the saving with pruned encodings at the ratio c . Then*

$$0.002095 < \sup_{c > 10} S(c) \leq 0.002096,$$

and every $c > 10$ with $S(c) \geq 0.002095$ lies in $(21, 23)$.

Proof. The lower bound is Proposition 6.5(iii). For the upper bounds, every $\theta \in (0, 0.9)$ gives $\Gamma(c) \leq \max(\gamma_X(c, \theta), \gamma_Q(\theta))$ by Lemma 6.1, and the values of γ_X , γ_Q and A_{pr} at rational arguments are enclosed between rational numbers by enclosing the logarithms that occur in them. On a cell $[c_1, c_2]$ both A_{pr} and Γ are nondecreasing, $F_A(\gamma)$ decreases in A and increases in γ , so

$$S(c) \leq F_{A_{\text{pr}}(c_1)}(\Gamma(c_2)) \quad (c_1 \leq c \leq c_2).$$

The interval $(10, 200]$ is cut into 222 cells, each with a certified bound of this form: 68 cells cover $(10, 21]$ and $[23, 200]$ with the bound 0.002095, and 154 cells, of width about 0.007 near the maximum, cover $[21, 23]$ with the bound 0.002096. For $c \geq 200$, $\Gamma(c) < 2/3$ and $A_{\text{pr}}(c) \geq A_{\text{pr}}(200) > 221.77$ give $S(c) < F_{A_{\text{pr}}(200)}(2/3) < 0.002095$. $\square$

In Lean: `optimum_global`, from `savingF_pruned_lt_outside`, `savingF_pruned_lt_global_tight`, `mem_100_of_savingF_pruned_ge` and `SstarPruned_bounds`; the supremum is `SstarPruned`. The cells are generated by the script `search/certs.py` of the repository, which chooses the endpoints and the points θ in floating point and checks every emitted inequality in exact rational arithmetic; Lean checks them again (files `Optimum/GlobalCells1.lean` to

`Optimum/GlobalCells5.lean`). The enclosures of the logarithms are those of [2], a series for $\ln \frac{1+z}{1-z}$ with an explicit remainder (`log_mem`).

The same method certifies the optimum with the paper’s encodings, which is the optimum of Theorem 7.4.

Theorem 6.7 (The optimum over the ratio, paper’s encodings). *Let $S(c) = F_{A_{\text{full}}(c)}(\Gamma(c))$ be the supremum of the saving with the paper’s encodings at the ratio c . Then*

$$0.002059 < \sup_{c > 10} S(c) \leq 0.002061,$$

and every $c > 10$ with $S(c) \geq 0.002059$ lies in $(20.8, 22)$.

Proof. As for Theorem 6.6, with A_{full} in place of A_{pr} : the lower bound is Proposition 6.5(i), and 261 cells cover $(10, 200]$, 87 below 20.8 and 86 above 22 with the bound 0.002059, and 88 on $[20.8, 22]$ with the bound 0.002061; for $c \geq 200$, $A_{\text{full}}(200) > 238.74$. $\square$

In Lean: `SstarFull`, `SstarFull_bounds`, `SstarFull_le`, `mem_100_of_savingF_full_ge`, `savingF_full_lt_global_tight`, `lt_SstarFull_iff`, and the cells `Optimum/GlobalFullCells1.lean` to `Optimum/GlobalFullCells6.lean`.

The upper bounds 0.002096 and 0.002061 are not strict, and the maximizers are not located more precisely than the two windows: both functions are flat near their maxima, $S(21) \approx 0.002093$ and $S(23) \approx 0.002092$ with pruned encodings, and $S(20.8) \approx S(22) \approx 0.0020588$ with the paper’s encodings, just below 0.002059.

7. CONSEQUENCES FOR 3SUM AND APSP

The theorems of this section are statements about programs of the word RAM, in the sense of Section 2.1. Four are for Exact Triangle and its consequences; they add the ingredients one at a time, and the first adds none. The fifth is the all-edges route to APSP.

7.1. From Exact Triangle to the three problems.

Proposition 7.1 ([2]). *Let $0 \leq \delta \leq 1$, and suppose that a program solves Exact Triangle on n vertices per part with weights up to n^ν in time $O(\nu n^{3-\delta}(\log n)^e)$, uniformly in ν . Then **ThreeSum.SolvedInTime** a holds for every rational $a > 2 - \delta/2$, and **MinPlusProduct.SolvedInTime** a and **APSP.SolvedInTime** a hold for every rational $a > 3 - \delta/3$.*

These are Theorem 21(a),(b) of the paper as formalized upstream (`threeSum_of_uniform`, `minPlus_polylog_of_uniform`, `apsp_polylog_of_uniform`). The hypothesis is obtained from the bound of Theorem 5.2, which holds from some size on, by solving smaller instances by brute force (`exactTriangleUniform_of_explicitFrom`).

7.2. The regime test. The upstream solver of the thin product tests whether $D^{18} \leq N$; it runs the recursion if so and brute force if not. The test is part of the program text, and it restricts the ratios at which the program can be used.

- The recursion must be applicable in the whole regime $D^{18} \leq N$: the tile $\sqrt{K}N_0$ must fit into N . For all large m this holds if $A_{\text{pr}}(c) < 18 \ln 4$, that is, for $c < 21.86 \dots$ (`exists_fits18`).
- The instances with $D \leq N^\varepsilon$ must lie in the regime: $\varepsilon \leq 1/18$. If $A_{\text{full}}(c) \geq 18 \ln 4$, that is, for $c \geq 21.49 \dots$, then $R_c(\gamma) \leq R_c(0) \leq 1/18$, and every admissible ε qualifies (`thinR_le_inv`).

Both conditions hold at $c = 108/5$ (`basePruned_108_5_lt`, `baseFull_ge_18`), and the numerals of Theorems 7.2 and 7.3, which use the program as it is, are obtained there. The optimum $c \approx 21.38$ of the paper's encodings lies just below this range: there ε has to exceed $1/18$ slightly, while $A_{\text{pr}}(c)/\ln 4 \approx 17.6$.

For Theorem 7.4 we therefore add a test with a rational exponent. Three procedures are appended to the upstream program (`programRS`): a test $D^r \leq N^s$ for natural numbers r, s (`testRSBody`: it computes N^s by a loop and decides $D^r < N^s + 1$ with upstream's routine `powLtBody`, never forming a number above $N^s D + N^s + D + r + s$), and a copy of the dispatching procedure that calls it. The analysis of the solver is carried out once for an abstract regime (`RegimeTest`, `thinClaimR_of_offlineWithin`), of which the upstream test and the new one are two instances (`regime18`, `regimeRS`). With the new test the two conditions above become $A_{\text{pr}}(c) < (r/s) \ln 4$ and $\varepsilon \leq s/r$ (`exists_fitsRS`, `thinClaimRS_gammaX`). Such a fraction exists for every admissible ε : $\varepsilon < R_c(\gamma)$ means $1/\varepsilon > (A_{\text{full}}(c) + \gamma \ln 4)/\ln 4 > A_{\text{pr}}(c)/\ln 4$, since $A_{\text{pr}} \leq A_{\text{full}}$ (`basePruned_le_baseFull`). The only remaining condition on the ratio is $\varepsilon \leq 1/16$, needed by the arithmetic of Theorem 5.2; $A_{\text{full}}(c) \geq 16 \ln 4$ for $c \geq 20$ gives it (`baseFull_ge_16`).

7.3. The four theorems.

Theorem 7.2 (The paper's exponents with better parameters). *With the program, the reduction and the leaf count of the paper, at $c = 108/5$ and suitable rational θ, ε and η : Exact Triangle is solved in time $O(n^{3-0.00184})$. Consequently `ThreeSum.SolvedInTime` holds with the exponent 1.99908, and `MinPlusProduct.SolvedInTime` and `APSP.SolvedInTime` with the exponent 2.99939.*

Nothing is new here except the choice of parameters. Corollary 26 of the paper rounds ε down to $1/18$ and takes $c = 21$; with $\gamma = \min(\gamma_{\text{pap}}(c, \theta), \frac{1}{2} - q(\theta))$ balanced and ε close to $R_c(\gamma)$ the saving $\varepsilon\gamma/2$ exceeds 0.00184 (`t2_witness`). In Lean: the general statement is `allSolved_paper`, from `thinClaim_gammaOf`, `hostBound_rat` and `explicitFrom_original`; the saving is `allSolved_paper_num`, and the end statements are `exactTriangle_paper`, `threeSum_paper`, `minPlus_paper` and `apsp_paper`.

Theorem 7.3 (With the exact count). *With the same program, analyzed by Theorem 3.5: Exact Triangle is solved in time $O(n^{3-0.00196})$. Consequently `ThreeSum.SolvedInTime` holds with the exponent 1.99902, and `MinPlusProduct.SolvedInTime` and `APSP.SolvedInTime` with the exponent 2.99935.*

In Lean: `allSolved_exact`, from `thinClaim_gammaX` and the same reduction; the witness is `t3_witness`, the saving `allSolved_exact_num`, and the end statements are `exactTriangle_exact`, `threeSum_exact`, `minPlus_exact` and `apsp_exact`.

Theorem 7.4 (With the true middle size). *Let $c \geq 20$ and $0 \leq \delta < F_{A_{\text{full}}(c)}(\Gamma(c))$. With the host of Theorem 5.1, the thin product of Theorem 3.5 and the regime test with a rational exponent, Exact Triangle is solved in time $O(n^{3-\delta})$; consequently so is every δ below $\sup_{c>10} F_{A_{\text{full}}(c)}(\Gamma(c)) \in (0.002059, 0.002061]$. At $c = 107/5$ this holds for $\delta = 0.002059$. Consequently `ThreeSum.SolvedInTime` holds with the exponent 1.99898, and `MinPlusProduct.SolvedInTime` and `APSP.SolvedInTime` with the exponent 2.99932.*

Proof. By Theorem 6.2 and Proposition 6.3 there is an admissible rational tuple $(c', \theta', \sigma', \varepsilon')$ near c with saving above δ . Lower γ below $\gamma_X(c', \theta')$ and ε to a rational so that the saving stays above δ (`exists_gamma_rat_of_lt`), and choose r/s between $A_{\text{pr}}(c')/\ln 4$ and $1/\varepsilon'$ (Section 7.2). Theorem 3.5 with the new test gives the thin product, and Theorem 5.2 with $\eta = 1 - 1/(2\sigma)$ and $d = \varepsilon/(1 - \eta)$ slightly lowered gives Exact Triangle. Its second condition holds because $\frac{3}{2}d \leq 3\sigma\varepsilon$

and $\delta < \varepsilon(1 - \sigma)$, so $\frac{3}{2}d + \delta < (1 + 2\sigma)\varepsilon < 3\varepsilon \leq \frac{3}{16}$. The supremum over c is Theorem 6.7, and the value at $c = 107/5$ is Proposition 6.5(i). $\square$

In Lean: `allSolved_full` is the statement for every $c \geq 20$, `allSolved_full_sup` the statement for every saving below the supremum `SstarFull`, `allSolved_full_num` the saving at $c = 107/5$, and the end statements are `exactTriangle_full`, `threeSum_full`, `minPlus_full` and `apsp_full`. The five-digit exponents are those of $\delta = 0.00205$: the optimum improves the saving at $c = 108/5$ by less than $2 \cdot 10^{-7}$.

Theorem 7.5 (With pruned encodings). *Let $c \geq 21$ and $0 \leq \delta < F_{A_{\text{pr}}(c)}(\Gamma(c))$. With the host of Theorem 5.1 and the thin product of Theorem 4.6, Exact Triangle is solved in time $O(n^{3-\delta})$; consequently so is every δ below $\sup_{c \geq 10} F_{A_{\text{pr}}(c)}(\Gamma(c)) \in (0.002095, 0.002096]$. At $c = 22$ this holds for $\delta = 0.002095$. Consequently *ThreeSum.SolvedInTime* holds with the exponent 1.99896, and *MinPlusProduct.SolvedInTime* and *APSP.SolvedInTime* with the exponent 2.99931.*

The proof is that of Theorem 7.4 with R'_c in place of R_c and Theorem 4.6 in place of Theorem 3.5; the test with a rational exponent is part of the program `programP`. For $c \geq 21$ we have $A_{\text{pr}}(c) \geq 16 \ln 4$ (`basePruned_ge_16`), so $\varepsilon \leq 1/16$ and $\frac{3}{2}d + \delta < 3\varepsilon \leq 0.19$. The supremum over c is Theorem 6.6. At $c = 22$ the parameters are $\Gamma > 0.0762$, $\theta \approx 0.116$, $\sigma \approx 0.519$ and ε just below $R > 0.05498$ (`Gam_22_bounds`, `thinR_pruned_bounds`, `savingF_pruned_gt`). In Lean: `allSolved_pruned` is the statement for every $c \geq 21$ (file `Pipeline/PrunedClaim.lean`; the form that still takes the thin-product claim as an argument is `allSolved_pruned_of_claim`, from `explicitFrom_pruned_of_claim`), `allSolved_pruned_sup` the statement for every saving below the supremum `SstarPruned`, `allSolved_pruned_num` the saving at $c = 22$, and the end statements are `exactTriangle_pruned`, `threeSum_pruned`, `minPlus_pruned` and `apsp_pruned`. No hypothesis is involved.

7.4. APSP through all-edges Exact Triangle. Footnote 10 of the paper observes that APSP can keep all of the saving of Exact Triangle instead of a third: "the proof of Theorem 17 also solves the all-edges version of Exact Triangle (scanning each pair only until its zero triangle is found), to which the (min, +)-product reduces on the same n vertices". We carry this out as programs. The *all-edges version* of Exact Triangle asks, for every pair (a, b) of the two outer parts, whether some c makes $w(a, b) + w(b, c) + w(a, c) = 0$; as a task of the machine (`aeTask`) it writes the n^2 answers to an output array (`aeFlags`).

Lemma 7.6 (Comparisons as equalities). *Let $0 \leq x, y, u < 2^b$ be integers and $f(\ell) = \lfloor u/2^\ell \rfloor - \lfloor x/2^\ell \rfloor - \lfloor y/2^\ell \rfloor$. Then*

$$x + y < u \iff f(0) = 1 \text{ or } f(\ell) \in \{2, 3\} \text{ for some } \ell \leq b.$$

Proof. $f(b) = 0$, and $f(\ell) = 2f(\ell + 1) + e_\ell$, where $e_\ell \in \{-2, -1, 0, 1\}$ is the bit of u at position ℓ minus those of x and y . So $f(\ell + 1) \geq 2$ implies $f(\ell) \geq 2$. If $f(\ell) \in \{2, 3\}$ for some ℓ , then $f(0) \geq 2$ by descent, and $f(0) = u - x - y$. Conversely let $f(0) \geq 1$. If $f(0) \neq 1$, let ℓ be the largest index with $f(\ell) \geq 2$; then $\ell < b$, $f(\ell + 1) \leq 1$ and $f(\ell) \leq 2f(\ell + 1) + 1 \leq 3$. $\square$

In Lean: `add_lt_iff_shiftDiff`.

Proposition 7.7. *Let A, B be $n \times n$ integer matrices with entries of absolute value at most W , let $2W < 2^b$, and let t be a matrix of thresholds with $-2W \leq t_{ij} < 2^b - 2W$. For $\ell \leq b$ put $X_\ell(i, k) = \lfloor (A_{ik} + W)/2^\ell \rfloor$, $Y_\ell(k, j) = \lfloor (B_{kj} + W)/2^\ell \rfloor$ and $U_\ell(i, j) = \lfloor (t_{ij} + 2W)/2^\ell \rfloor$. Then, for every pair (i, j) , there is a k with $A_{ik} + B_{kj} < t_{ij}$ if and only if, for one of the $2b + 3$ pairs (ℓ, r) with $(\ell, r) = (0, 1)$ or $r \in \{2, 3\}$, there is a k with*

$$X_\ell(i, k) + Y_\ell(k, j) + (r - U_\ell(i, j)) = 0.$$

Consequently a binary search that runs for all pairs (i, j) in parallel computes the $(\min, +)$ -product of A and B from the answers to $R(2R + 3)$ all-edges Exact Triangle instances on the same n vertices per part, where $R = \lceil \log_2(4W + 1) \rceil$.

Proof. Apply Lemma 7.6 to $x = A_{ik} + W$, $y = B_{kj} + W$ and $u = t_{ij} + 2W$. The entry C_{ij} of the product is the unique integer $c \in [-2W, 2W]$ such that the answer is negative for the threshold c and positive for $c + 1$; a binary search finds it in R rounds, and a round asks one matrix of thresholds. $\square$

In Lean: `exists_lt_iff_exists_exactTriangle`, `minPlus_unique`, `search_eq_minPlus` and `search_exactTriangle`.

The reduction of the paper (Theorem 21(b), [10]) ends with a binary search of exactly this kind, over a procedure that answers the threshold questions for all pairs at once (`pairsTask`, `isHost_mp`), followed by repeated squaring for APSP (`claim_apspFromMinPlus`). The factor 3 is lost before that, in a blocking of size $n^{1/3}$ that reduces the all-pairs questions to the decision problem. Two hosts replace the blocking.

Theorem 7.8 (All pairs from all-edges Exact Triangle). *There is a program that, given a procedure solving all-edges Exact Triangle within $T(n, U)$ steps, answers the threshold questions of Proposition 7.7 for all pairs within $(2b + 3)(T(n, 9U) + O(n^2)) + O(n^2)$ steps, where $b = \lceil \log_2 3U \rceil + 1$; in particular within $O((1 + \log U)(T(n, 9U) + n^2))$.*

Proof. With $W = U$ the hypotheses of Proposition 7.7 hold, and the weights of the instances are at most $2^b + 3 \leq 9U$. The program writes $A + U$, the transpose of $B + U$ and $t + 2U$, and then runs the b levels from the top: at each level it asks the two instances with $r = 2, 3$ and takes the disjunction of their answers into the output, then halves every number (upstream's pass for $\lfloor x/2^\ell \rfloor$, which avoids division); at level 0 it also asks $r = 1$. The output is then the matrix of answers. $\square$

In Lean: `isHost_pairsAE` with the time `pairsTimeAE`, from the bridge `pairFlags_iff` between the two tasks and the invariant `orFlags_eq_pairFlags`; the bound `pairsTimeAE_le` with the constant `CA`, and the transfer `pairsFromAllEdges`.

Theorem 7.9 (Theorem 5.1 for the all-edges version). *Theorem 5.1 holds for all-edges Exact Triangle in place of Exact Triangle, with the same bound and another constant C .*

Proof. The host of Theorem 17 keeps n^2 flags, one per pair (a, b) , zero at the start. For every query pair that an instance accepts and whose flag is still 0, it scans the piece C_k of the instance and stores the result in the flag; pairs whose flag is 1 are skipped. Every pair with a zero triangle is accepted, and hit, in the instance of the piece of its c , so the flags are the answers at the end. A scan either fails, which the paper charges to a false positive of the prime, or turns a flag from 0 to 1; so there are at most $F(p) + n^2$ scans. The n^2 additional scans cost $O(n^2 \sqrt{D'}/g')$, which is within the term $n^2 D' g'$; zeroing and copying the flags costs $O(n^2)$. $\square$

In Lean the pure part is `flagsAt_m` (the flags after all instances are the answers) and `sum_execsAE_le` (the count of the scans); the procedure that scans the accepted pairs is `scanPairsAEBody`, the brute force of the small case `bruteAEBody`, the host `ae17_solves`, its time `hostTimeAE` and `obeysBound17Mid_hostTimeAE`, and the claim `claim17MidAE_of_pack`. The statement of Theorem 5.1 is the same proposition `Claim17Mid` in a time model (`lightModelAE`) whose reading of "Exact Triangle" is the all-edges task; the deductions of Sections 5 and 7 are made for an arbitrary model, so they apply unchanged (`MidHostRat`, `midHostRat_ae`, `explicitFrom_full`, `explicitFrom_pruned`).

Theorem 7.10 (APSP through all-edges Exact Triangle). *Let $c \geq 20$ and $0 \leq \delta < F_{A_{\text{full}}(c)}(\Gamma(c))$, or let δ be any number below $\sup_{c \geq 10} F_{A_{\text{full}}(c)}(\Gamma(c)) > 0.002059$. Then the (min, +)-product and APSP are solved in time $O(n^a)$ for every rational $a > 3 - \delta$; consequently **MinPlusProduct**, **SolvedInTime** and **APSP**. **SolvedInTime** hold with the exponent 2.99795. With pruned encodings the same holds for $c \geq 21$ and $\delta < F_{A_{\text{pr}}(c)}(\Gamma(c))$, for every $\delta < \sup_c F_{A_{\text{pr}}(c)}(\Gamma(c)) > 0.002095$, and with the exponent 2.99791.*

Proof. Theorems 7.4 and 7.5 with Theorem 7.9 in place of Theorem 5.1 give all-edges Exact Triangle in time $O(n^{3-\delta} \log n)$ for weights up to $n^{O(1)}$. Theorem 7.8 and the search of Proposition 7.7 (upstream’s program, $O(\log U)$ rounds at the bound $6U$) give the (min, +)-product of matrices with entries up to u in time $O(\log^2 u (T(n, 54u) + n^2))$, and repeated squaring gives APSP with $O(\log n)$ products of matrices with entries up to $O(nu)$. Along $u = n^\kappa$ every factor is $n^{3-\delta}$ times a polylogarithm. $\square$

In Lean: `minPlusSolved_full`, `minPlusSolved_full_sup`, `minPlusSolved_pruned` and `minPlusSolved_pruned_sup` (the structure `MinPlusSolved`), through `solvedIn_mp_of_allEdges`, `minPlusInPolylog_of_explicitFrom`, `apspInPolylog_of_explicitFrom` and `minPlusSolved_of_explicitFrom`; the end statements are `minPlus_allEdges`, `apsp_allEdges`, and with pruned encodings `minPlus_allEdges_pruned`, `apsp_allEdges_pruned`, the strongest statements of the paper for the (min, +)-product and APSP.

8. THE FORMALIZATION

The Lean development is the directory `ImprovedExponents` of the repository <https://github.com/qawbecrdtey/ImprovedExponents>, which also contains this paper. It has 179 files and 46 416 lines, of which 26 822 are generated certificates for Theorems 6.6, 6.7 and 9.1; about 19 600 lines are written by hand.

8.1. Relation to the upstream formalization. The part of the formalization [2] of the paper that we use, the 340 files of Lean that our development imports directly or transitively, is included in the repository at a fixed revision (`e1a4e65`; directory `upstream/3sum-apsp`) and ported from Lean and Mathlib `v4.33.1`, for which it was written, to `v4.35.0-rc2`. The port changes only what no longer compiles, in 4 of the files, and no statement or definition; every other file is upstream’s, byte for byte. It provides the word `RAM` and the notion `Problem.SolvedInTime` (file `EndStatement.lean`), a small programming language with a time model (`lightModel`) and a compiler to the word `RAM`, the programs of the paper, and the proofs of its theorems.

We reuse the following.

- The program of Section 4, `program31`, for arbitrary rational parameters, with its correctness and the bound of its running time in terms of the two costs of Theorem 30.
- The count of the boxes, `lemma_29_count`, which leaves the tail $\sum_{d \geq t} \beta_d$ explicit.
- The host of Theorem 17 and the passage from a host to a claim about programs, `claim17_of_host`.
- The reductions of Theorem 21 and the passage to the end statements: `threeSum_of_uniform`, `minPlus_polylog_of_uniform`, `apsp_polylog_of_uniform` and `SolvedInTime.endStatement`.
- The enclosures of logarithms between rational numbers, `log_mem`.

Upstream proves the bounds of the paper at the parameters of the paper. Where an upstream proof fixes parameters, we repeat it for general ones: the solver of the thin product at arbitrary ratios (`Pipeline/ThinClaim.lean`), the time of the program against the exact tail (`Cost8X`), the routines that compute the parameters of the host (`ParamRoutines`), and the arithmetic of

the total time (**Total**). Where an upstream proof assumes the full encodings, we repeat it under the weaker contract of Section 4.4: the encoder of Theorem 5, the procedures of Theorem 30 and the routines with chosen parameters (**PrunedProgram**), and the time of the new program against **cost8P** (**Cost8P**). The file **NOTICE** of the repository lists the files that are adapted from upstream, which is distributed under the Apache 2.0 license.

8.2. **What is new.** Table 8.1 lists the directories.

directory	lines	content
ExactCount	763	g_2 , Theorem 3.3, Proposition 3.6
Cost8X	727	the upstream program against the exact tail (Theorem 3.5)
PrunedEncoding	1330	Theorem 4.2, Proposition 4.5
PrunedProgram	3951	the pruned encoder and the solver under the weaker contract (Section 4.4)
Cost8P	718	the new program against cost8P (Theorem 4.6)
HostMid	1008	the host with one added assignment (Theorem 5.1)
ParamRoutines	645	routines for $\lfloor n^{a/b} \rfloor$ and $\lceil D^{c/d} \rceil$
Pipeline	2337	thin-product claims, the regime tests, the offline routines, the general theorems
Total	806	Theorem 5.2
Optimum	29 754	Section 6, Proposition 6.4, the shapes of Section 9; 26 822 lines are generated
MinPlus	400	Lemma 7.6, Proposition 7.7
AllEdges	3558	the all-edges route to APSP (Theorems 7.8, 7.9, 7.10)
Statements.lean	396	the theorems of Section 7

TABLE 8.1. The Lean development.

Two points of the formalization differ from a proof on paper.

The programs are almost fixed. In Theorems 7.2 and 7.3 the program for the thin product and the host of the reduction are upstream’s, at other parameters; the only new program text is that of the routines that compute $D' = \lfloor n^{a/b} \rfloor$ and $g' = \lceil D^{c/d} \rceil$. So the improvement from 0.00175 to 0.00196 is obtained by a new analysis of existing programs. Theorem 7.4 adds one assignment to the host and three short procedures for the regime test; the all-edges route (Theorem 7.10) adds the two hosts of Section 7.4, which reuse upstream’s passes, scans, hashing and instance construction. Theorem 7.5 adds seven procedures (**programP**): the pruned encoder and six that call it, or call upstream’s procedures under the new contract; the procedures that build the tiles and answer the queries are upstream’s, with a new proof.

Numerals are corollaries. The general theorems (**allSolved_paper**, **allSolved_exact**, **allSolved_full**, **allSolved_pruned**) quantify over the parameters or over all savings below the closed form $S(c)$. The decimal exponents of Table 1.1 follow from certified enclosures of $S(c)$ at one ratio (Proposition 6.5).

8.3. **The main statements.** Table 8.2 lists the Lean names of the main statements.

8.4. **What has to be trusted.** For the end statements of Theorems 7.2, 7.3, 7.4, 7.5 and 7.10: the Lean kernel, and the definitions of upstream’s **EndStatement.lean**, which state what it means that a problem is solved in time $O(n^r)$ on the word RAM. That file imports nothing, and the module **ImprovedChallenge.lean** of the repository, which states the four strongest

statement	declaration	file
Theorem 3.3	sum_beta_le_exp	ExactCount/Tail.lean
Proposition 3.6	sum_card_Leaves_le_exp	ExactCount/Leaves.lean
Theorem 3.5	offlineWithinX	Cost8X/Offline.lean
Theorem 4.2	pruned_encoding	PrunedEncoding/Pruned.lean
Proposition 4.5	costsP	PrunedEncoding/Costs.lean
Remark 4.4	encWorkP_le_M_div	PrunedProgram/Work.lean
Theorem 4.6	prunedEncoderClaim	Pipeline/PrunedClaim.lean
	encodeP_spec, tEncP_le	PrunedProgram/Encode/Recursion.lean
	offline32P_meets	PrunedProgram/Wrapper/Offline.lean
	offlineWithinP	Cost8P/Offline.lean
Theorem 5.1	claim17Mid_of_paramProcs	HostMid/AtParameters.lean
Theorem 5.2	explicitFrom_mid	Total/Explicit.lean
Theorem 6.2	isLUB_tupleSaving	Optimum/PerCSupremum.lean
Proposition 6.3	exists_rat_tuple_basePruned	Optimum/PerCDensity.lean
Proposition 6.4	isLUB_paperSaving, savingF_GamHalf_lt	Optimum/PaperAccounting.lean
Theorem 6.6	optimum_global	Statements.lean
Theorem 6.7	optimum_global_full	Statements.lean
Theorem 7.2	threeSum_paper, apsp_paper	Statements.lean
Theorem 7.3	threeSum_exact, apsp_exact	Statements.lean
Theorem 7.4	allSolved_full, threeSum_full	Statements.lean
Theorem 7.5	allSolved_pruned	Pipeline/PrunedClaim.lean
	threeSum_pruned, apsp_pruned	Statements.lean
Lemma 7.6	add_lt_iff_shiftDiff	MinPlus/Comparison.lean
Proposition 7.7	search_exactTriangle	MinPlus/ExactTriangle.lean
Theorem 7.8	isHost_pairsAE	AllEdges/Pairs/Host.lean
Theorem 7.9	ae17_solves	AllEdges/Host/ProgramAE.lean
	midHostRat_ae	AllEdges/Host/ClaimAE.lean
Theorem 7.10	minPlusSolved_full, apsp_allEdges	Statements.lean
	minPlusSolved_pruned, apsp_allEdges_pruned	Statements.lean
Proposition 9.1	shapes_table	Optimum/Shapes/Table.lean

TABLE 8.2. The main statements and their Lean names. The end statements for Exact Triangle, the (min, +)-product and APSP are named in the same way (exactTriangle_full, minPlus_full, apsp_full, minPlus_allEdges, and so on).

end statements, contains its 132 lines of Lean verbatim and imports nothing either. An end statement such as

```
theorem threeSum_pruned : EndStatement.ThreeSum.SolvedInTime 1.99896
```

mentions no definition of ours and takes no argument. No statement of this paper rests on a hypothesis.

The development contains no `sorry` and declares no axiom. Lean reports that the end statements depend on the axioms `propext`, `Classical.choice` and `Quot.sound`, the three axioms that Mathlib uses throughout, and on no other; in particular the numerical certificates are checked by the kernel and not by compiled code.

8.5. How to check. In the root of the repository, `lake exe cache get` fetches the compiled Mathlib, and `lake build` checks everything: the included part of upstream, the library and the two modules described next; the script `scripts/check-upstream.sh` compares the included part with upstream’s revision and confirms that every changed file is marked and listed. The module `ImprovedChallenge.lean` states the four strongest end statements, `threeSum_`

`pruned` (1.99896), `exactTriangle_pruned` (2.99791), `minPlus_allEdges_pruned` and `apsp_allEdges_pruned` (2.99791; Theorems 7.5 and 7.10), with upstream’s `EndStatement.lean` copied into it verbatim (the script `scripts/check-endstatement.sh` confirms the copy against the included file, which is upstream’s) and nothing imported, each with `sorry` for its proof; the module `ImprovedSolution.lean` imports the proofs. The two modules and the configuration `comparator.json` are in the form that the tool `Comparator` [5] checks: that the solution proves the statements of the challenge, literally, that it uses only the axioms `propext`, `Quot.sound` and `Classical.choice`, and that everything replays in Lean’s kernel and in independent kernels; the compilations run in a sandbox. From Lean v4.35.0-rc2 on, `Comparator` ships with the toolchain as `lake comparator`, with the independent kernels `NanoDa` and `con-ron`. The script `scripts/verify-comparator.sh` runs it as the Palomar registry does, in a bubblewrap sandbox and in a fresh copy of the sources without build products, so that the challenge, the solution and everything they import are compiled inside the sandbox. We ran it on the final development: `lake comparator` accepted the pair on 2026-10-07 (“Your solution is okay!”, all three kernels; the log is `scripts/comparator-run.log`). Every Lean file of the repository uses Lean’s module system, and the statement module imports nothing, as the Palomar registry [6] requires of a submission.

The certificates of Theorems 6.6 and 6.7 are regenerated by `python3 -m search.certs`, those of Proposition 9.1 with the option `-encoding shapes`, and the script `check_lean_names.py` next to the source of this paper checks that every Lean name cited here exists in the sources.

8.6. What is not formalized. Only parts of Section 9: the discussion of mixtures of shapes, the grid of hypothetical shapes, the experiments on the arrangement of the wanted set, and the remark on ω . Each is marked “not formalized” there. Everything else, including the comparison of the shapes of Schönhage’s family (Proposition 9.1), is machine-checked.

9. LIMITS OF THE METHOD

The saving 0.002095 is the end of what the parameters give. This section records why the base identity cannot be exchanged for a better one of the same kind, and what would have to change. The comparison of the shapes of Schönhage’s family is certified (Proposition 9.1): it is a statement about the saving formula of Sections 3 to 6 evaluated for other shapes, since the programs exist for the shape of the paper only. The rest of the section, the mixtures, the hypothetical shapes, the experiments and the remark on ω , is not formalized; its numbers are computed in floating point by the scripts in the directory `search` of the repository.

9.1. The family of Schönhage. For $k, n \geq 2$, Schönhage’s identity for $\langle k, 1, n \rangle \oplus \langle 1, (k-1)(n-1), 1 \rangle$ has $kn+1$ terms [7]. It has $s = kn$ outer outputs with one term each, and one shared term for an inner product of length $b = (k-1)(n-1)$. The paper uses $k = n = 3$: $s = 9$, $b = 4$, ten terms. The analysis of Sections 3 to 6 is written for this shape. For another member of the family it would be repeated with the corresponding counts of terms, variables and leaves, an asymmetric shape being used together with its transpose so that tiles stay square; we do not write the programs for other shapes, and what follows is the saving formula of Theorem 6.2 evaluated for them. With s outer outputs and inner length b one has $D = b^m$, $N_0 = \sqrt{s}^{L-m}$, g_2 with $\delta \ln s$ in place of $\delta \ln 9$, γ_X and q with $\ln b$ in place of $\ln 4$, the window $(0, s/(s+1))$ for θ in place of $(0, 0.9)$, and the thinness constant $A_{\text{pr}}(c) = \frac{c}{2}H(\frac{1}{c}) + (c-1)\frac{\ln s}{2}$; the saving at the ratio $c > s+1$ is then $S(c) = F_{A_{\text{pr}}(c)}(\Gamma(c))$ as in Theorem 6.2. In Lean (directory `Optimum/Shapes`): `g2S`, `gammaXS`, `qS`, `gammaQS`, `GamS`, `thinRS`, `savingS`, `basePrunedS` and the saving `shapeSaving s b c`; for $(s, b) = (9, 4)$ they reduce to the functions of Sections 3 to 6 (`shapeSaving_nine_four`, from `g2S_nine`, `gammaXS_nine_four`,

`qS_nine_four`, `GamS_nine_four` and `basePrunedS_nine`). Lemma 6.1 and the monotonicity of A_{pr} carry over under $2 \leq b < (s+1)^2$ and $c > s+1$ (`GamS_pos`, `GamS_lt_two_thirds`, `GamS_monotoneOn`, `basePrunedS_monotoneOn`). Table 9.1 gives the optimum for each shape, with pruned encodings and the prime at its fixed point.

shape (k, n)	s	b	$\sup_c S(c)$ (certified)	best c	Γ	ε
(3, 3)	9	4	(0.002095, 0.002096]	21.92	0.0759	0.0552
(2, 4)	8	3	(0.001875, 0.001877]	20.11	0.0749	0.0501
(3, 4)	12	6	(0.001867, 0.001869]	27.96	0.0746	0.0501
(2, 5)	10	4	(0.001776, 0.001778]	24.24	0.0742	0.0479
(2, 3)	6	2	(0.001669, 0.001671]	16.03	0.0742	0.0450
(2, 6)	12	5	(0.001623, 0.001625]	28.39	0.0733	0.0443
(3, 5)	15	8	(0.001613, 0.001615]	34.04	0.0732	0.0441
(4, 4)	16	9	(0.001573, 0.001575]	35.96	0.0729	0.0432
(4, 5)	20	12	(0.001323, 0.001325]	44.01	0.0714	0.0371
(5, 5)	25	16	(0.001098, 0.0011]	54.02	0.0698	0.0315

TABLE 9.1. The saving for Exact Triangle with Schönage’s identity of shape (k, n) and pruned encodings. The enclosures of the suprema over the ratio $c > s+1$ are certified in Lean (Proposition 9.1; Theorem 6.6 for (3, 3)). The last three columns, the ratio at which the supremum is approached and the parameters there, are computed in floating point and are not certified.

Proposition 9.1 (The family of Schönage). *For each of the nine shapes of Table 9.1 other than (3, 3), the supremum over $c > s+1$ of the saving $S(c)$ of the shape lies in the interval given in the table; in particular it is below 0.00188. For (3, 3) the saving exceeds 0.002095 at $c = 22$.*

Proof. As for Theorem 6.6, shape by shape: a lower bound is the value at one rational ratio, and the upper bound is certified on cells covering $(s+1, c_{\max}]$, with $c_{\max}$ between 200 and 600, and beyond $c_{\max}$ by $\Gamma(c) < 2/3$ and the monotonicity of A_{pr} ; the cells are finest on the window around the maximum, which is [19.6, 20.6] for (2, 4) and [52.1, 56.1] for (5, 5). $\square$

In Lean: `shapes_table` collects the nine upper bounds and the value for (3, 3); the enclosures are `sSup_shapeSaving_S24_mem` and its analogues for the other shapes, from the per-shape bounds `shapeSaving_S24_lt` and `shapeSaving_S24_gt`. The 2253 cells, in 53 generated files `Optimum/Shapes/Cells_S24_1.lean` and so on, are produced by `search/certs.py` with the option `-encoding shapes` and checked by Lean’s kernel as in Theorem 6.6.

The shape (3, 3) is the best. The exponent Γ hardly varies over the family, and (3, 3) also has the largest ε . For the same reason mixtures do not help (not formalized): when two shapes are used on different levels, the optimizer drives the share of the second shape to zero, because no trade-off between Γ and ε is available.

9.2. Hypothetical shapes. This subsection is not formalized; its numbers are floating point. One can ask what an identity with s outer outputs, one term each, and a shared term of inner length b would give, whether or not it exists. The saving grows with b and falls with s . For $s = 9$ it is 0.00210 at $b = 4$, 0.00254 at $b = 5$ and 0.00293 at $b = 6$; for $s = 8$ it is 0.00188 at $b = 3$ and 0.00252 at $b = 4$. In the computed grid ($5 \leq s \leq 12$, $3 \leq b \leq 8$) every cell with a larger saving than $(s, b) = (9, 4)$ is outside the class of Schönage’s construction: an identity of this kind with $s = kn$ satisfies the pairing bound $b \leq (k-1)(n-1)$, so $s = 9$ allows $b \leq 4$, $s = 8 = 2 \cdot 4$ allows $b \leq 3$, $s = 6$ allows $b \leq 2$, $s = 10$ allows $b \leq 4$, $s = 12$ allows $b \leq 6$, and

$s = 5, 7$ have no factorization. So within the class of identities with one term per outer output, the identity of the paper is optimal for this method. Even the realistic cells beyond the bound ($s \in \{8, 9\}$, $b \in \{4, 5\}$) give savings of only 0.0025 to 0.0031, that is, 3SUM in about $n^{1.9985}$.

9.3. Experiments on the arrangement of the wanted set. This subsection is not formalized. The bound (2.1) is a worst case over the wanted sets. A random wanted set of the same size meets it up to lower-order factors: a leaf of order d is shared by at most $\binom{L-m+d}{d}$ output strings, and the expected number of visited leaves of order d is about $\min(|U|\alpha_d, \beta_d)$. A gain needs a wanted set that is close to a union of sub-tiles. Simulation cannot test this: feasible sizes have $c = L/m \approx 3$, where the β_d grow with d and every wanted set saturates the tree, while the relevant regime $c \approx 21$ has at least 10^{21} leaves already for $m = 1$. Zeros in the encodings do not help either: an encoding at a leaf with P_0 on a set Z of levels is a signed sum of at least $3^{|Z|}$ entries, and for the dominant leaves ($|Z| \approx 0.89m$) the expected number of nonzero terms tends to infinity.

9.4. Open problems.

- (i) *New sharing structures.* An identity in which outer outputs are fed by several terms, or with several inner outputs, is outside the leaf counting of Section 2.2. Scoring such an identity needs a generalization of α_d and β_d .
- (ii) *Structured wanted sets.* The reductions from 3SUM produce Exact Triangle instances whose weights are array entries at additively combined indices. Whether the wanted sets are then far from random is not known.

Remark 9.2 (not formalized). The exponent of matrix multiplication plays no role. The choice of the prime costs $n^\omega D^{3/2}$, which is of lower order with Strassen's $\omega \leq \log_2 7$ already; rectangular fast matrix multiplication would not change any exponent of this paper.

Acknowledgments. The algorithms, the reductions and the framework of thin products are those of Alman and Vassilevska Williams [1]; this paper changes their analysis and their algorithm in a few places, and it would not exist without their work, nor without the Lean formalization [2] of their paper by Anthropic, on which the development builds. The proofs, the Lean development and the text of this paper were written by the language model Claude Fable 5.1 (Anthropic), run as an agent in Claude Code, under the direction of the author, who chose the goals, decided how the results are stated and reviewed the outputs, and wrote no Lean by hand. The repository <https://github.com/qawbecrdtey/ImprovedExponents> contains the development and this paper, and records the checks that were run (Section 8).

REFERENCES

1. Josh Alman and Virginia Vassilevska Williams, *Truly subquadratic 3SUM and truly subcubic APSP via triangles in sparse lopsided graphs*, 2026, [arXiv:2610.06783](https://arxiv.org/abs/2610.06783).
2. Anthropic, *Truly subquadratic 3SUM and truly subcubic APSP: a Lean 4 formalization*, Directory 3sum-apsp of <https://github.com/anthropics/formal-math>, 2026.
3. Timothy M. Chan and Qizheng He, *Reducing 3SUM to convolution-3SUM*, Proc. 3rd SIAM Symposium on Simplicity in Algorithms (SOSA 2020), 2020, pp. 1–7.
4. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021.
5. Lean FRO, *Comparator: checking a Lean solution against a trusted statement*, <https://github.com/leanprover/comparator>, with the exporter `lean4export` (<https://github.com/leanprover/lean4export>) and the independent kernel `nanoda` (https://github.com/robsimmons/nanoda_lib), 2026.
6. Palomar Registry, *Palomar: a registry of Lean-verified mathematics*, <https://palomar-registry.org/>, 2026.

7. Arnold Schönhage, *Partial and total matrix multiplication*, SIAM Journal on Computing **10** (1981), no. 3, 434–455.
8. Volker Strassen, *Gaussian elimination is not optimal*, Numerische Mathematik **13** (1969), 354–356.
9. The mathlib Community, *The Lean mathematical library*, Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020.
10. Virginia Vassilevska Williams and R. Ryan Williams, *Subcubic equivalences between path, matrix, and triangle problems*, Journal of the ACM **65** (2018), no. 5, 27:1–27:38.
11. Virginia Vassilevska Williams and Ryan Williams, *Subcubic equivalences between path, matrix and triangle problems*, Proc. 51st IEEE Symposium on Foundations of Computer Science (FOCS 2010), 2010, pp. 645–654.
12. ———, *Finding, minimizing, and counting weighted subgraphs*, SIAM Journal on Computing **42** (2013), no. 3, 831–854.
13. Frank Yates, *The design and analysis of factorial experiments*, Technical Communication 35, Imperial Bureau of Soil Science, Harpenden, 1937.

Email address: qawbecrdtey@kaist.ac.kr